



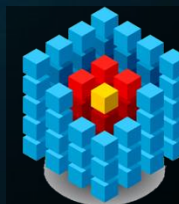
Inteligencia Artificial y Aprendizaje Automático

Sesión 6: Optimización de
CNNs y Redes recurrentes

Presentan:

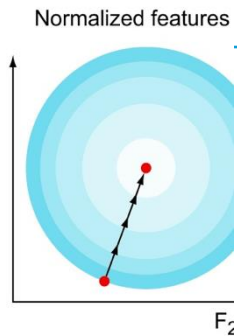
Dr. Ulises Olivares Pinto

Fernando Ramírez González



HPC
DATA LAB

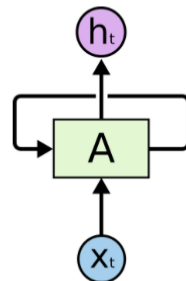
Contenido



Normalización de batch



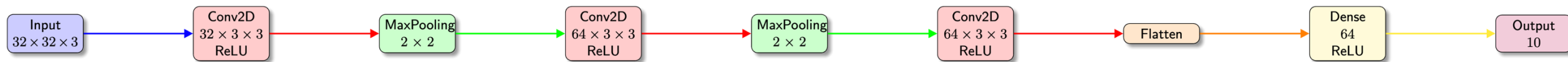
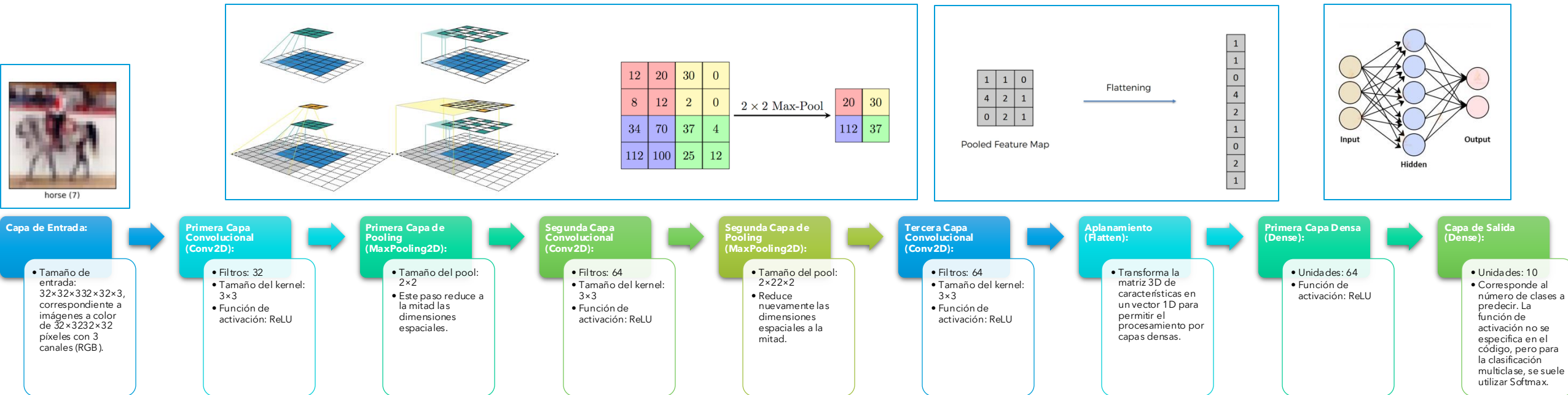
Transferencia de aprendizaje



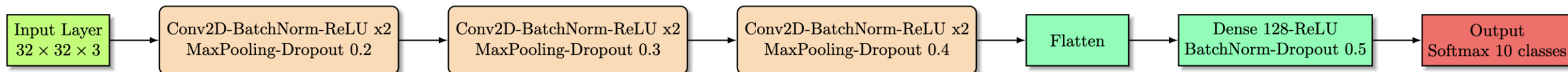
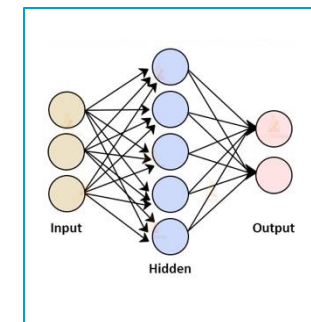
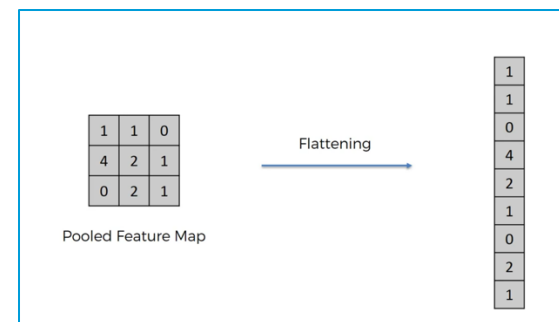
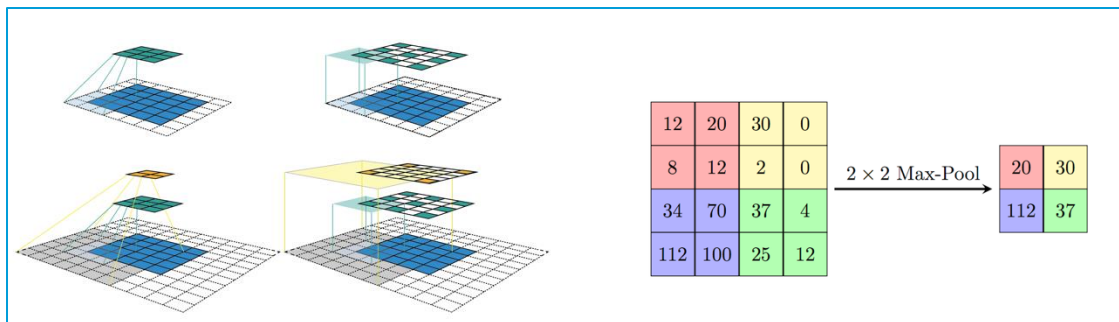
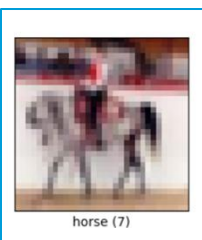
Redes Neuronales Recurrentes

- GRU
- LSTM
- Procesamiento de lenguaje natural

Arquitectura Simple



Arquitectura Mejorada



```
model = models.Sequential([
    # Capa de entrada y primera capa convolucional
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    # Segunda capa convolucional
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),

    # Tercera capa convolucional
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.4),

    # Capa de aplanamiento y capas densas
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
```

Arquitectura Mejorada

- Capa convolucional ("relu")
 - Conv2D
 - BatchNormalization
 - Conv2D
 - Batch Normalization
 - MaxPooling2D
 - Dropout
- Aumento de datos



Normalización de Batch



Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe
Google Inc., sioffe@google.com

Christian Szegedy
Google Inc., szegedy@google.com

Abstract

Training Deep Neural Networks is complicated by the fact that the distribution of each layer's inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to use much higher learning rates and be less careful about initialization. It also acts as a regularizer, in some cases eliminating the need for Dropout. Applied to a state-of-the-art image classification model, Batch Normalization achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. Using an ensemble of batch-normalized networks, we improve upon the best published result on ImageNet classification: reaching 4.9% top-5 validation error (and 4.8% test error), exceeding the accuracy of human raters.

Using mini-batches of examples, as opposed to one example at a time, is helpful in several ways. First, the gradient of the loss over a mini-batch is an estimate of the gradient over the training set, whose quality improves as the batch size increases. Second, computation over a batch can be much more efficient than m computations for individual examples, due to the parallelism afforded by the modern computing platforms.

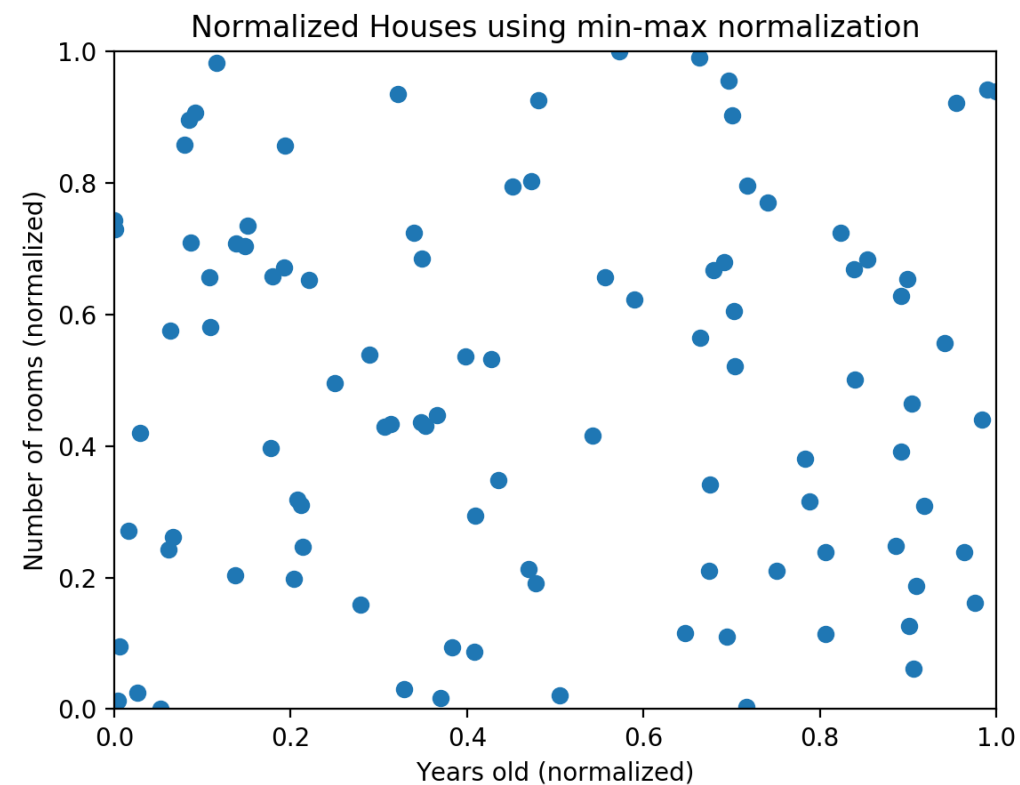
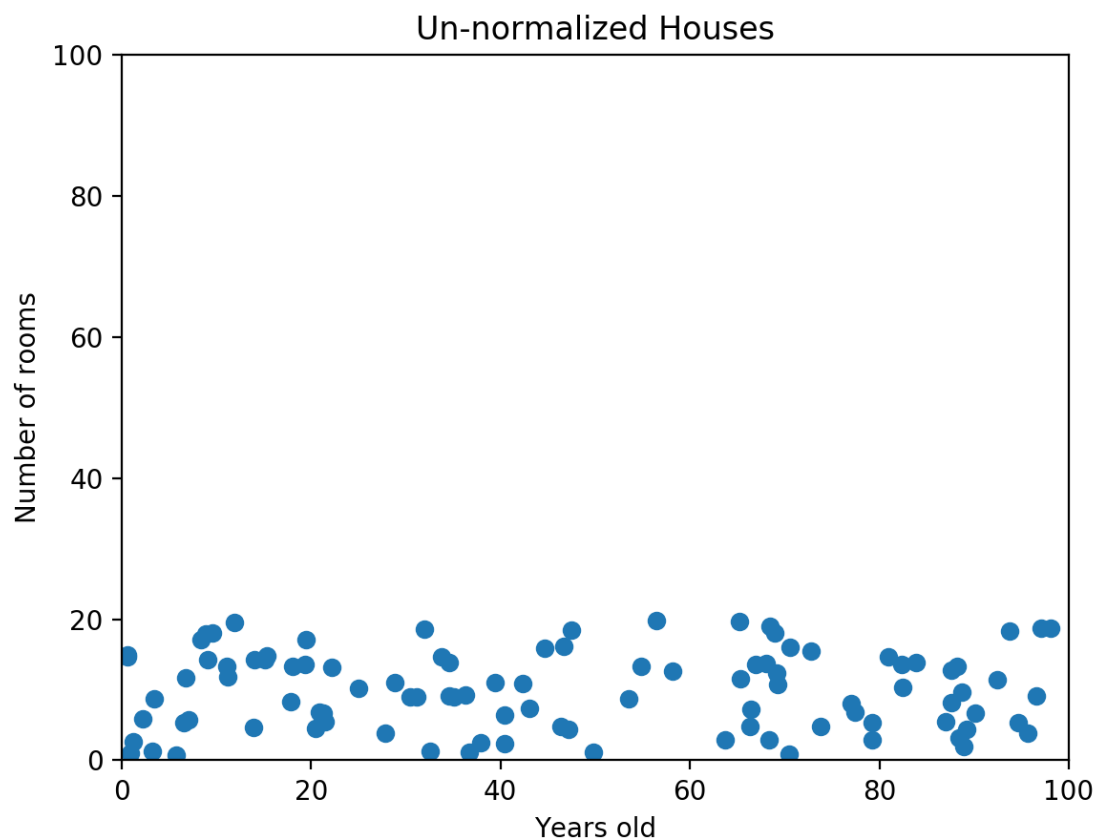
While stochastic gradient is simple and effective, it requires careful tuning of the model hyper-parameters, specifically the learning rate used in optimization, as well as the initial values for the model parameters. The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.

The change in the distributions of layers' inputs presents a problem because the layers need to continuously adapt to the new distribution. When the input distribution to a learning system changes, it is said to experience *covariate shift* (Shimodaira, 2000). This is typically handled via domain adaptation (Jiang, 2008). However, the notion of covariate shift can be extended beyond the learning system as a whole, to apply to its parts, such as a sub-network or a layer. Consider a network computing

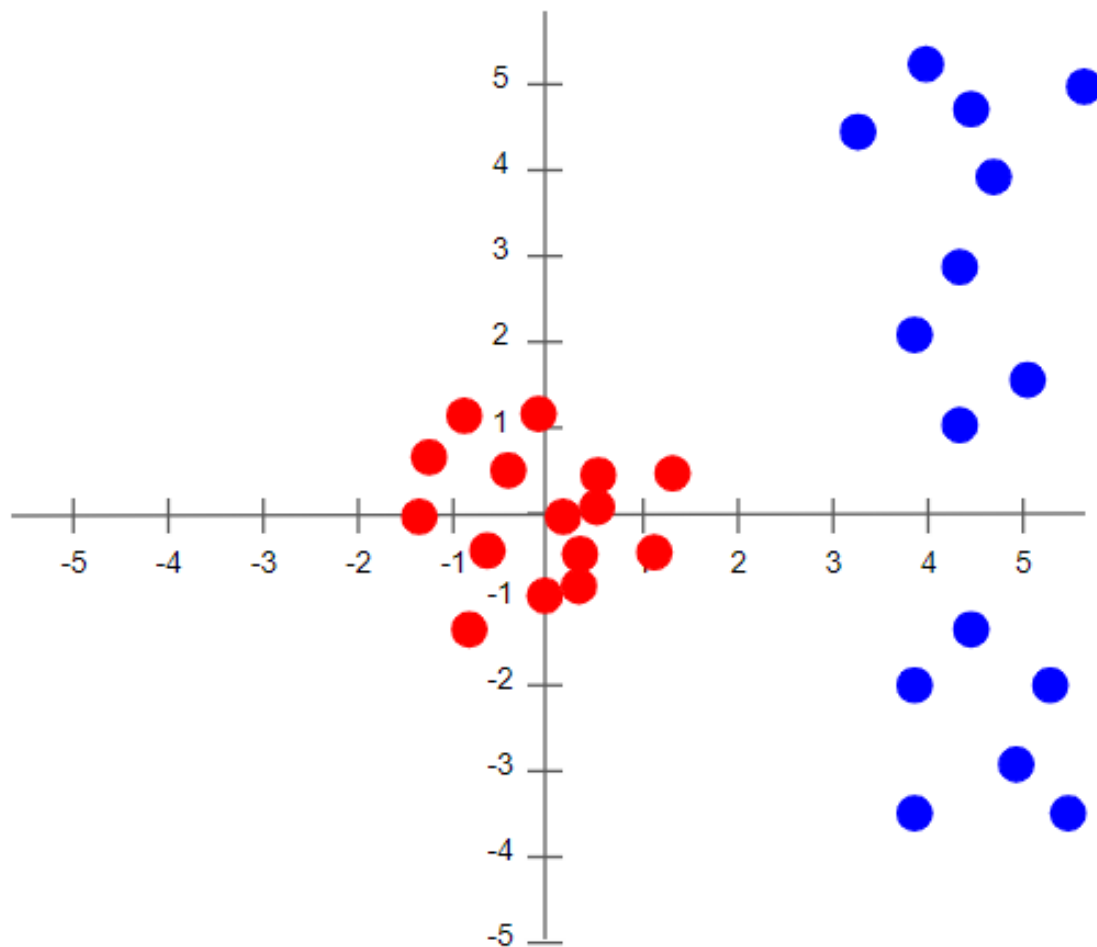
1 Introduction

$$\ell = F_2(F_1(u, \Theta_1), \Theta_2)$$

Normalización de Datos



Normalización de Datos

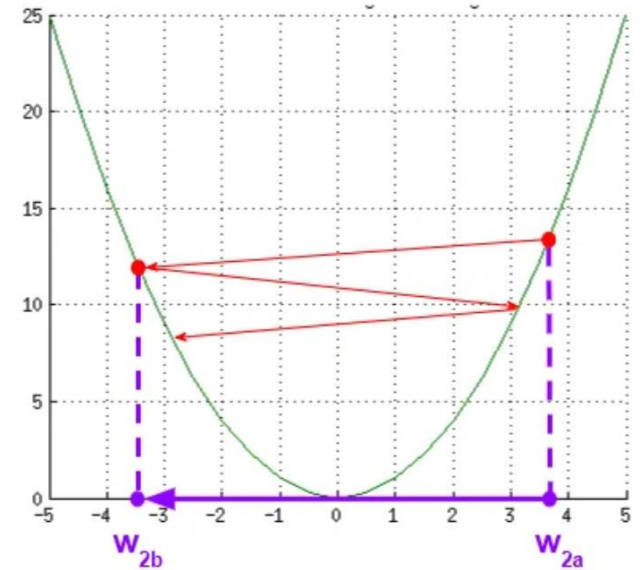
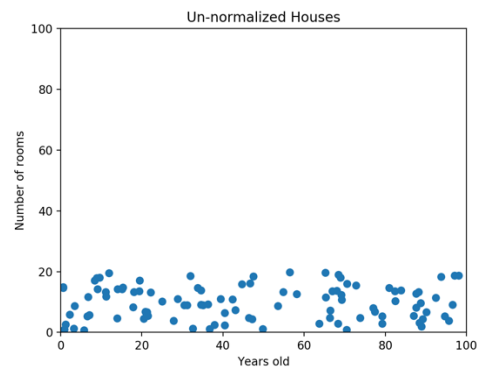
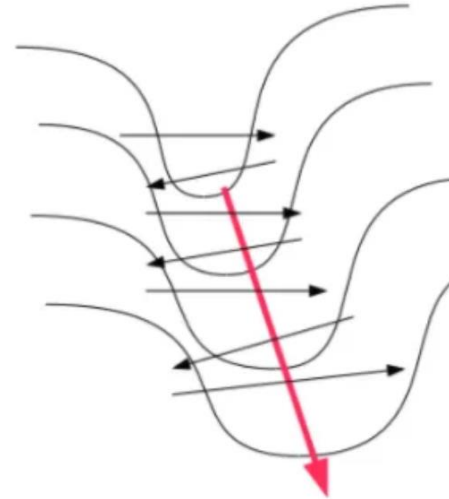
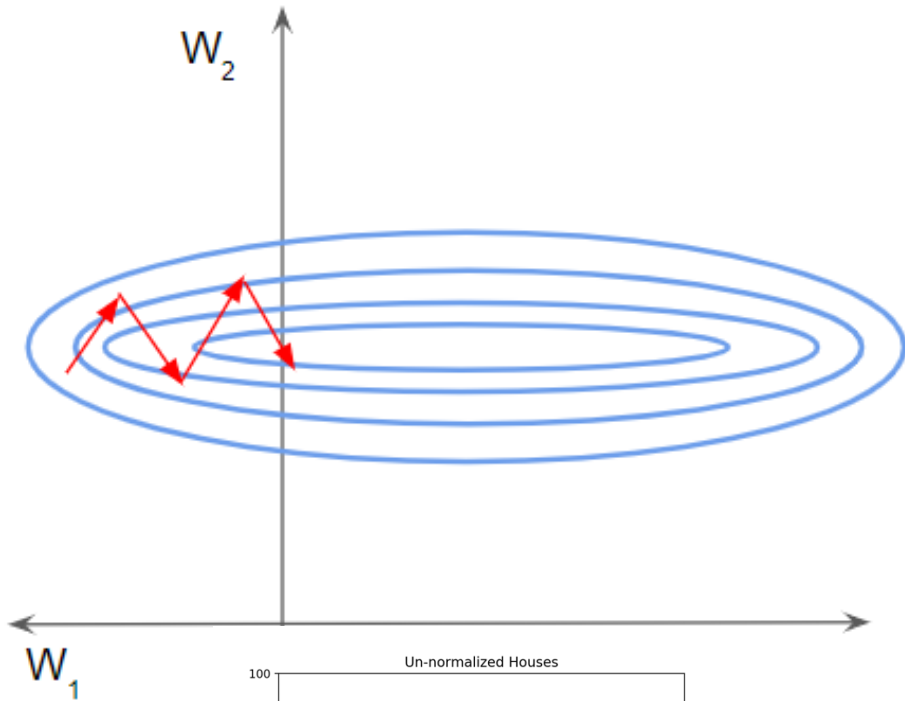


- **Datos Originales**

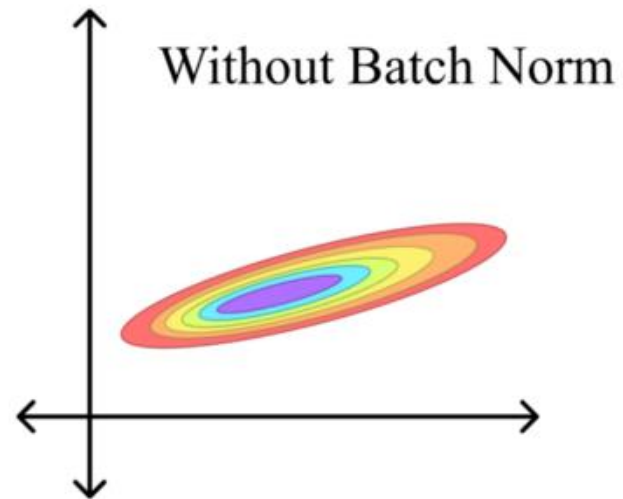
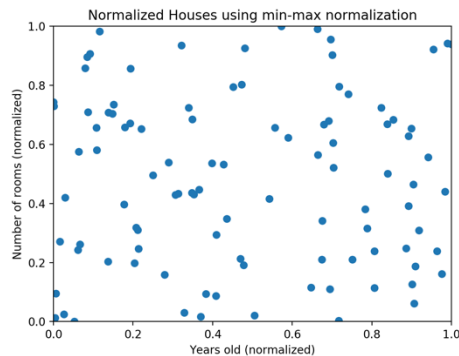
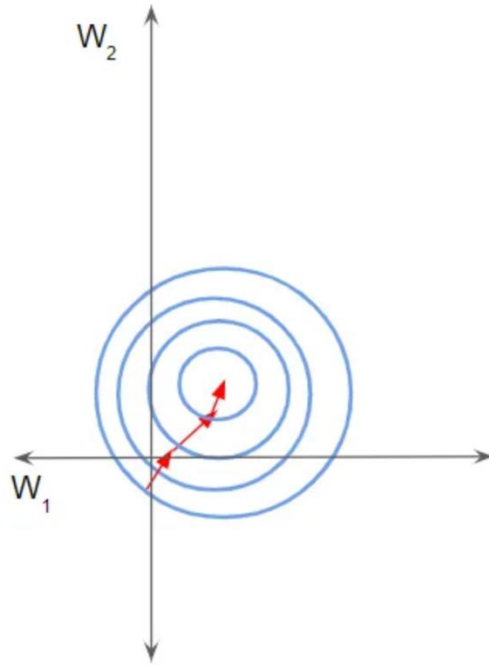
- **Datos Normalizados $[-1, 1]$**



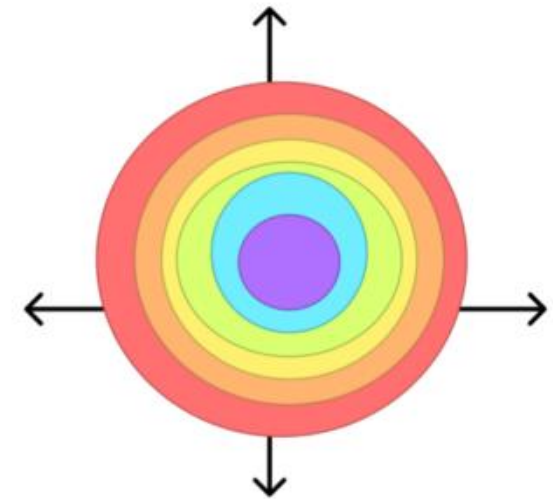
Proceso de entrenamiento de un modelo



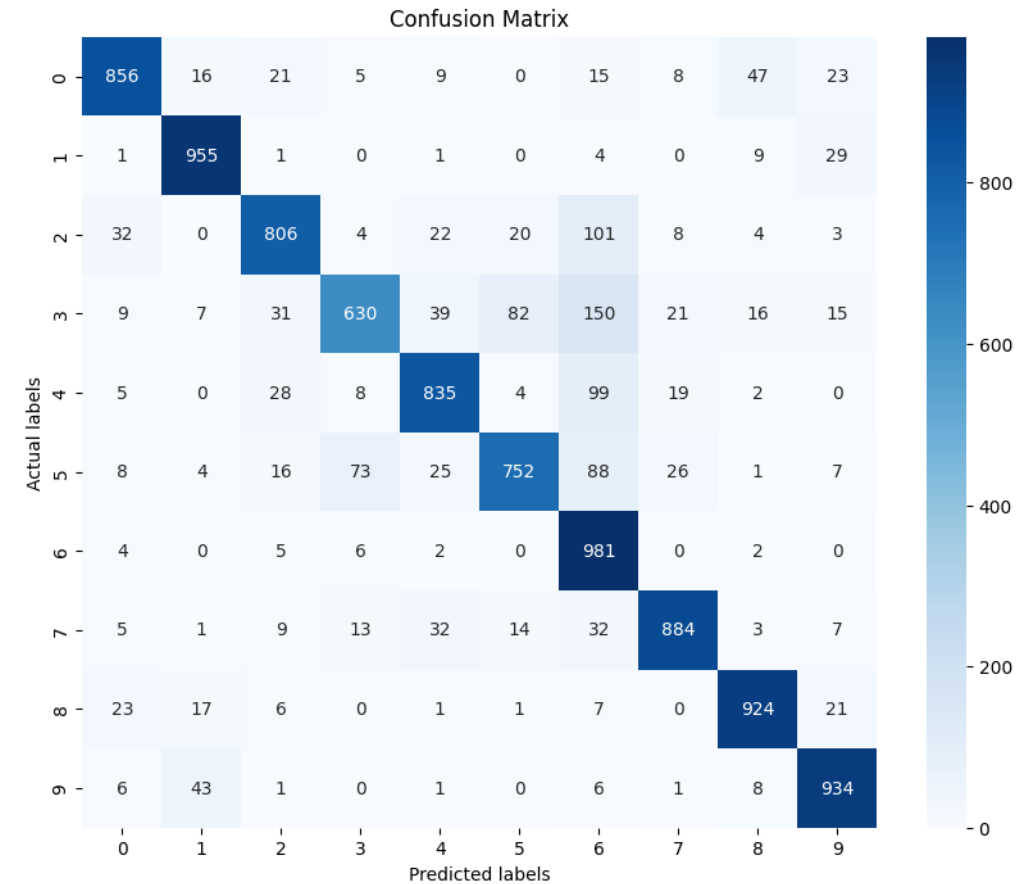
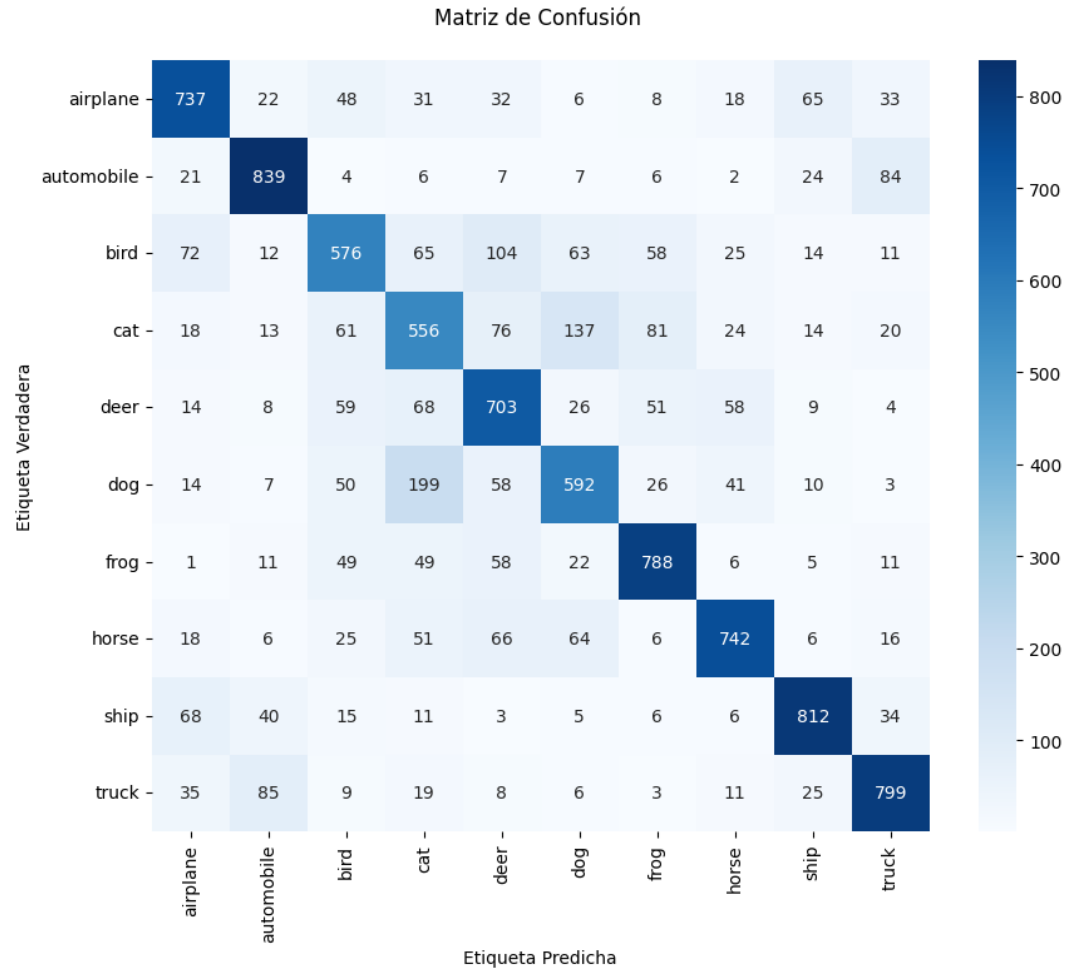
Normalización de Batch



With Batch Norm



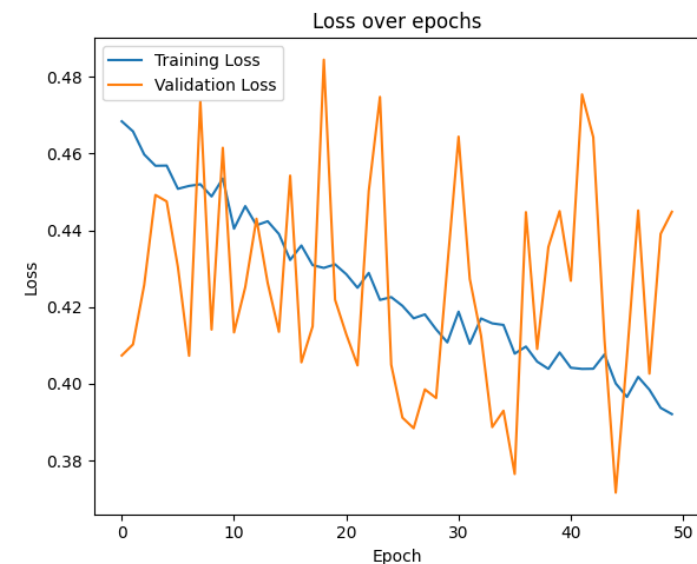
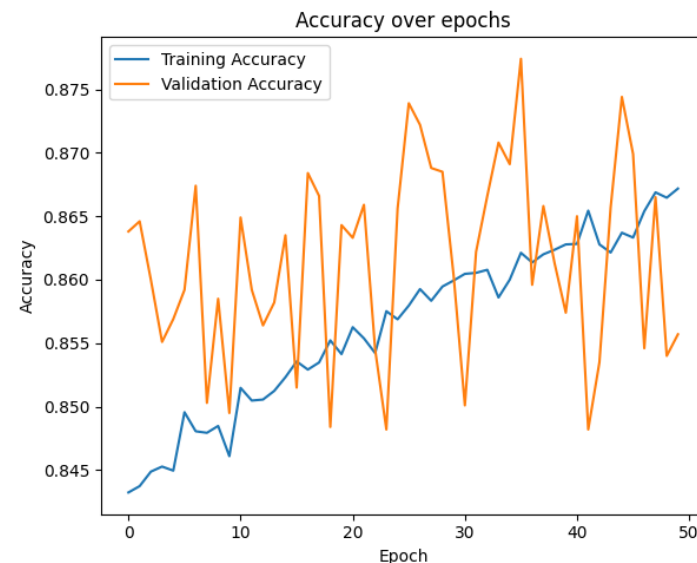
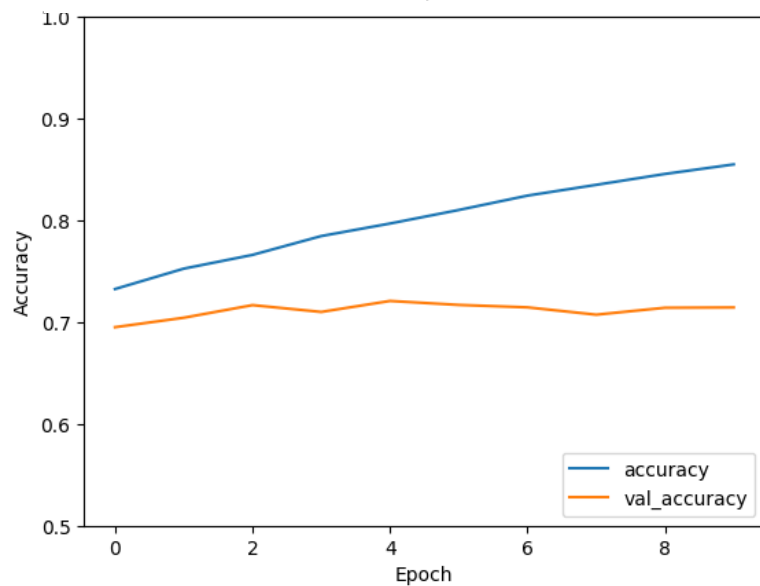
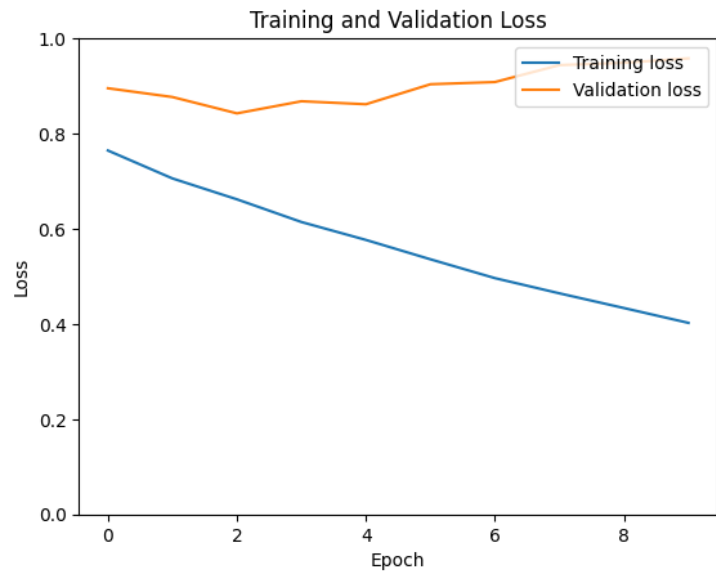
Evaluación y comparativa



313/313 - 4s - loss: 0.9580 - accuracy: 0.7144 - 4s/epoch - 13ms/step
presición sobre el set de datos de prueba: 0.7143999934196472

313/313 - 1s - loss: 0.4338 - accuracy: 0.8614 - 985ms/epoch - 3ms/step
Test Accuracy: 86.14%

Comparativa



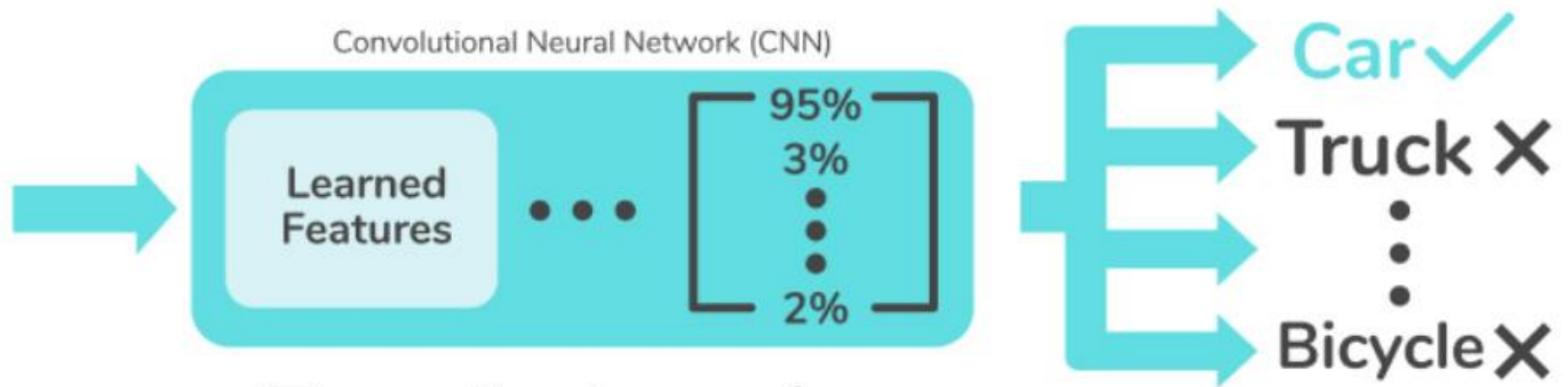
Transfer learning

Transfer Learning es una técnica poderosa que permite **reutilizar** modelos preentrenados para resolver tareas específicas sin necesidad de entrenar un modelo desde cero.



I know kung fu.

Training from Scratch



Transfer Learning



New data

Datasets



(b)



Proceso de para realizar Transfeer Learning en Tensor Flow



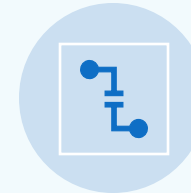
Selecciona un Modelo

Preentrenado: Elige un modelo preentrenado de TensorFlow que se ajuste a tu tarea. TensorFlow ofrece varios modelos preentrenados, como Inception, MobileNet, ResNet, etc. Puedes encontrar estos modelos en la [biblioteca TensorFlow Hub](#).



Carga el Modelo

Preentrenado: Utiliza TensorFlow para cargar el modelo preentrenado en tu proyecto. Puedes hacerlo usando `tf.keras.applications` para modelos populares o `tf.keras.utils.get_file` para modelos de TensorFlow Hub.



Personaliza las Capas

Superiores: Retira las capas superiores del modelo preentrenado (las capas de clasificación) y reemplázalas con capas personalizadas adecuadas para tu tarea. Por ejemplo, para la clasificación de imágenes, puedes agregar capas totalmente conectadas y una capa de salida con la cantidad correcta de neuronas para tus clases.

Código de ejemplo

Congela Capas Base (Opcional): En algunos casos, puedes optar por congelar las capas del modelo preentrenado (estableciendo `base_model.trainable = False`) para que no se actualicen durante el entrenamiento de Transfer Learning. Esto es útil cuando tienes un conjunto de datos pequeño y no deseas cambiar drásticamente los pesos preentrenados.

Entrena el Modelo: Utiliza tu conjunto de datos específico para entrenar el modelo. Puedes emplear un optimizador, una función de pérdida y métricas adecuadas a tu tarea.

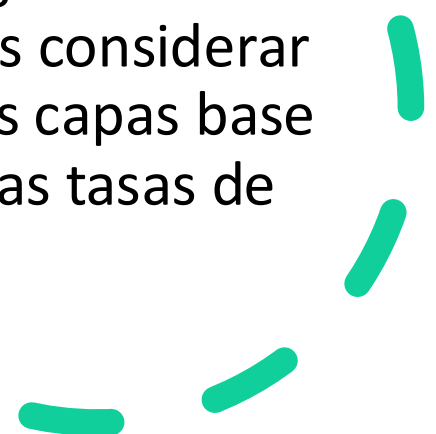
```
1 base_model = tf.keras.applications.MobileNetV2(input_shape=(224, 224, 3), include_top=False)
2 base_model.trainable = False
3
4 model = tf.keras.Sequential([
5     base_model,
6     tf.keras.layers.GlobalAveragePooling2D(),
7     tf.keras.layers.Dense(128, activation='relu'),
8     tf.keras.layers.Dense(num_classes, activation='softmax')
9 ])
```

```
1 model.compile(optimizer='adam',
2               loss='categorical_crossentropy',
3               metrics=['accuracy'])
4
5 model.fit(train_data, epochs=10, validation_data=validation_data)
```

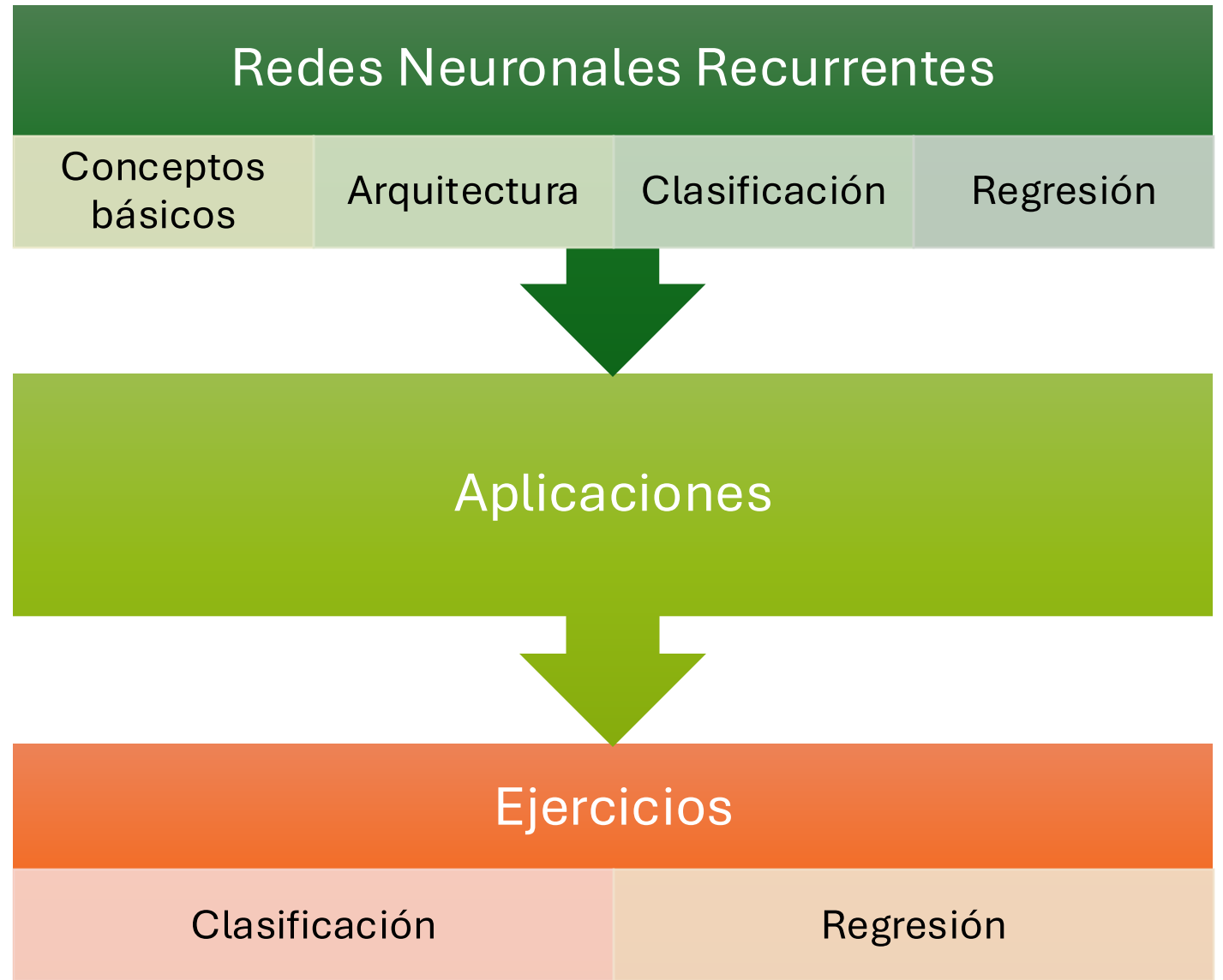
```
base_model.trainable = True
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

Fine-Tuning:

- Si tienes un conjunto de datos más grande o deseas afinar el rendimiento, puedes considerar el fine-tuning. Habilita algunas de las capas base para que sean entrenables y ajusta las tasas de aprendizaje si es necesario.

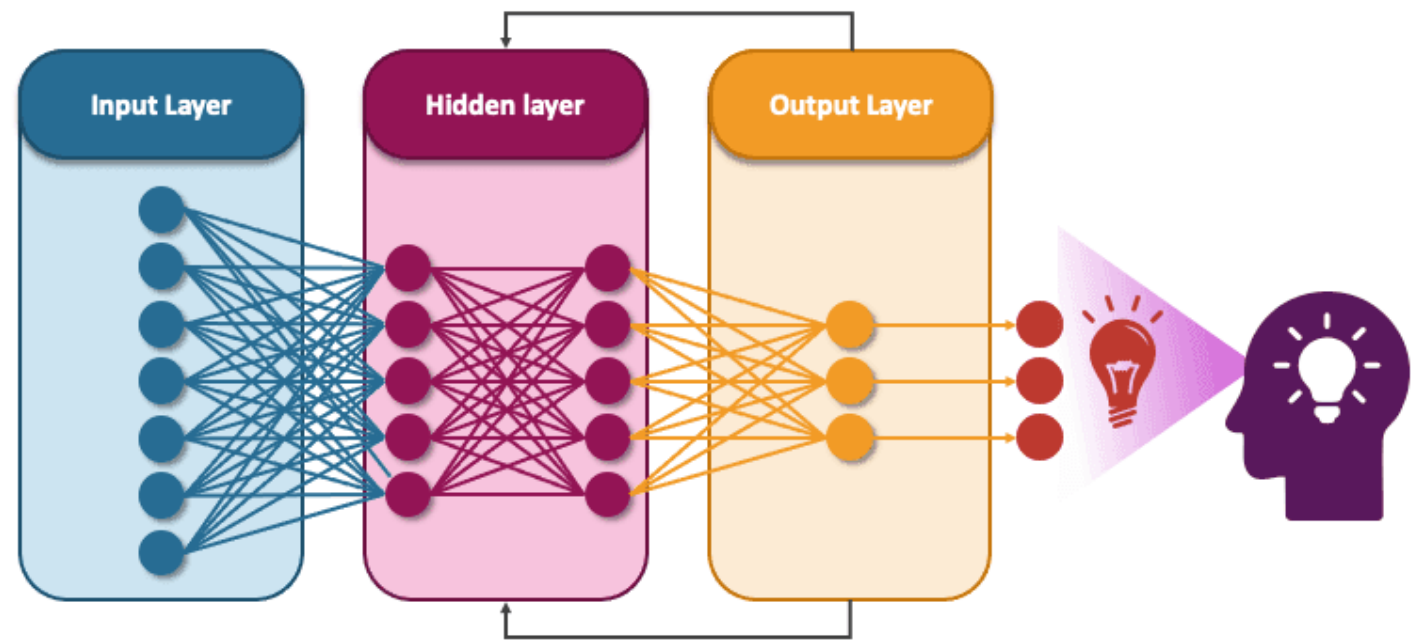


Contenido

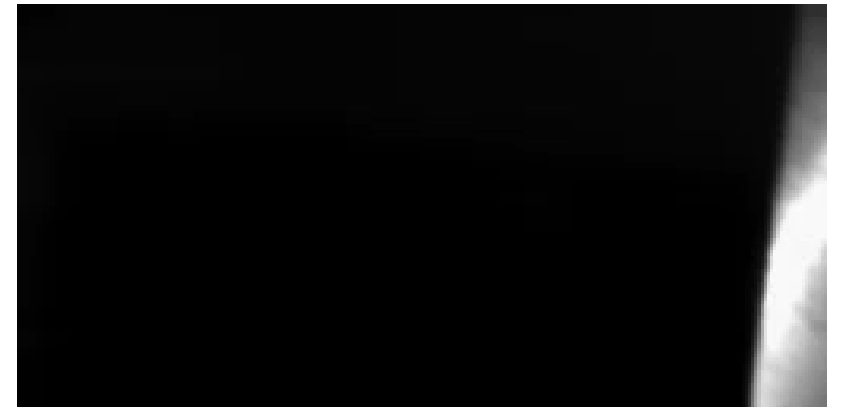


Redes Neuronaes Recurrentes

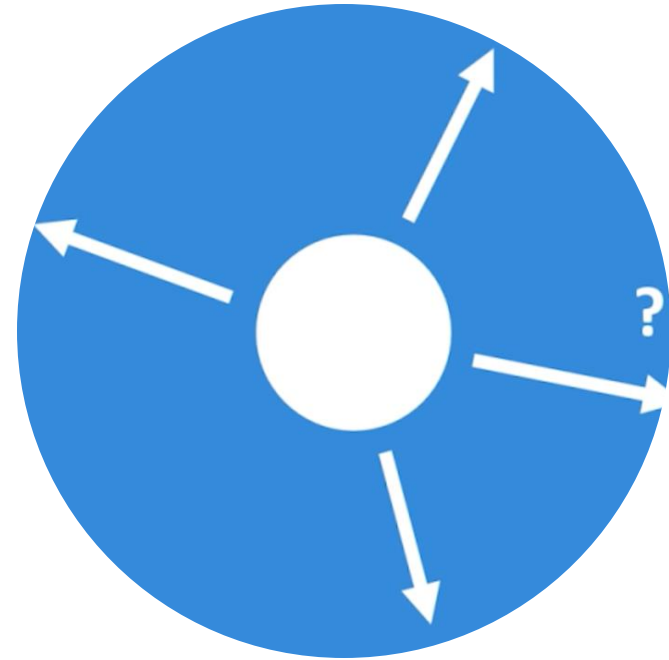
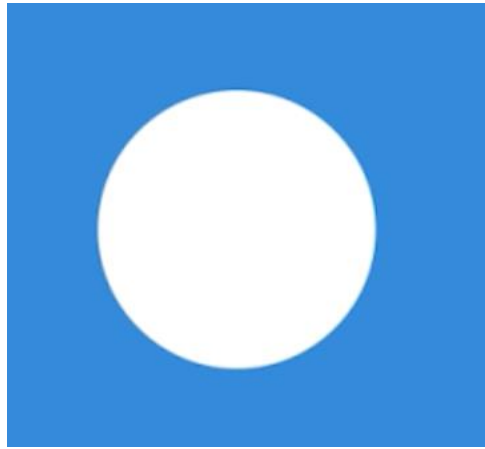
Deep Learning



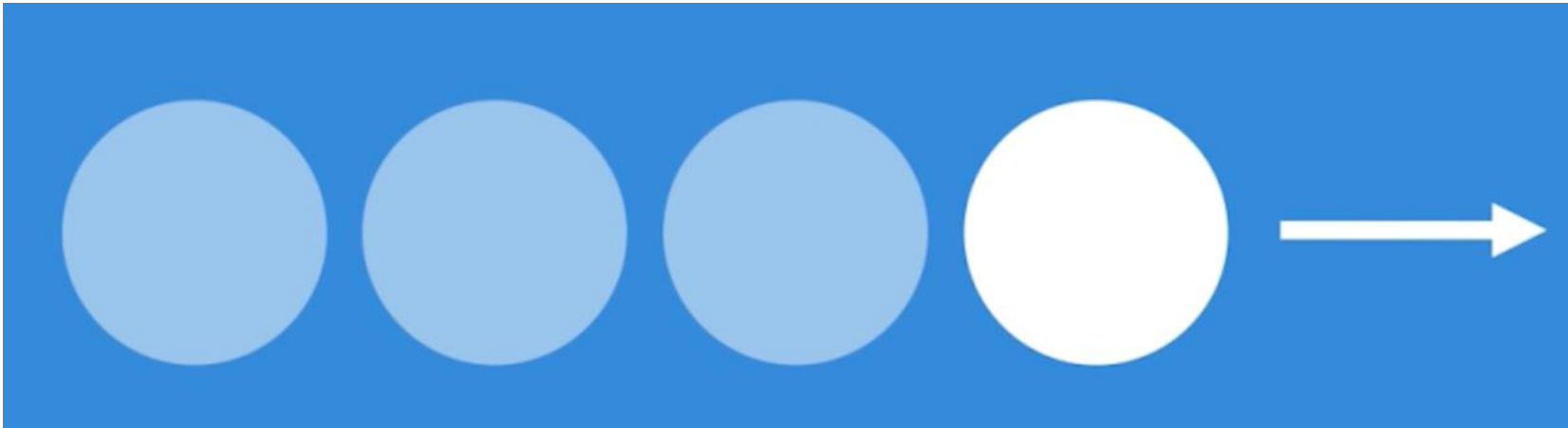




Dada la siguiente imagen
¿Es posible predecir hacia donde se moverá?



Dada la siguiente imagen
¿Es posible predecir hacia donde se moverá?



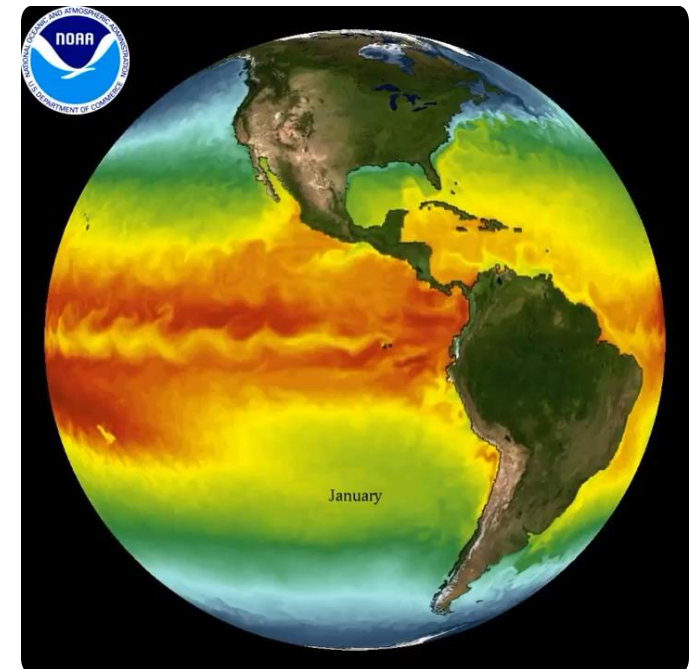
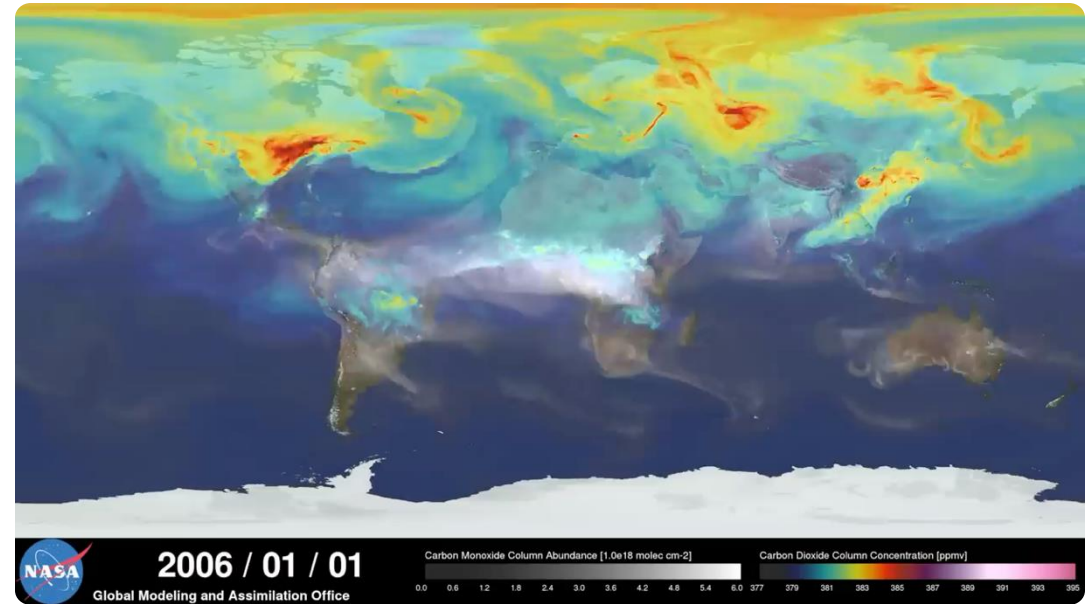
Datos secuenciales



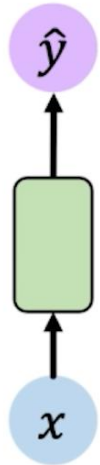
08, 2019

Secuencias en el mundo real

Secuencias en la vida real



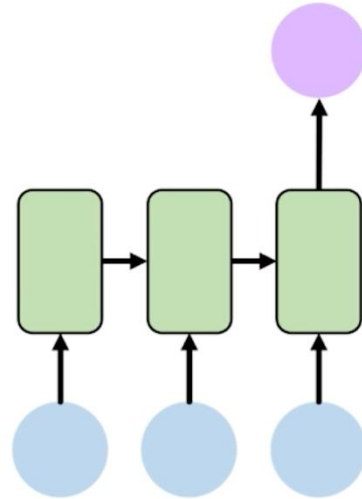
Datos Secuenciales



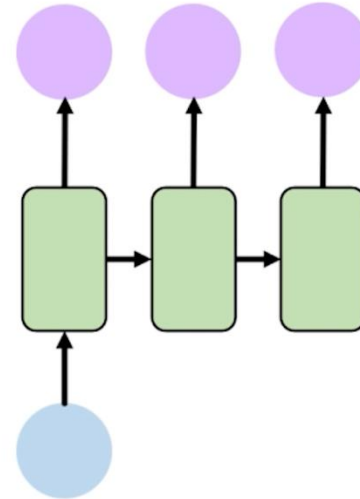
One to One
Binary Classification



“Will I pass this class?”
Student → Pass?



Many to One
Sentiment Classification

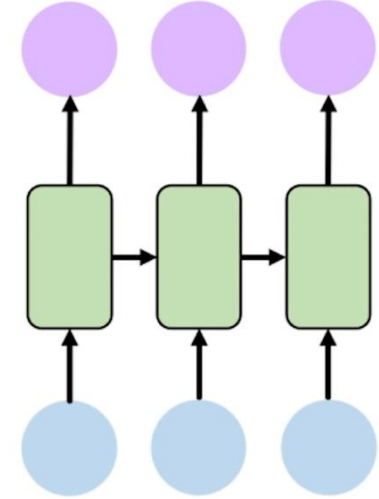


One to Many
Text Generation
Image Captioning

Image Captioning



“A baseball player throws a ball.”

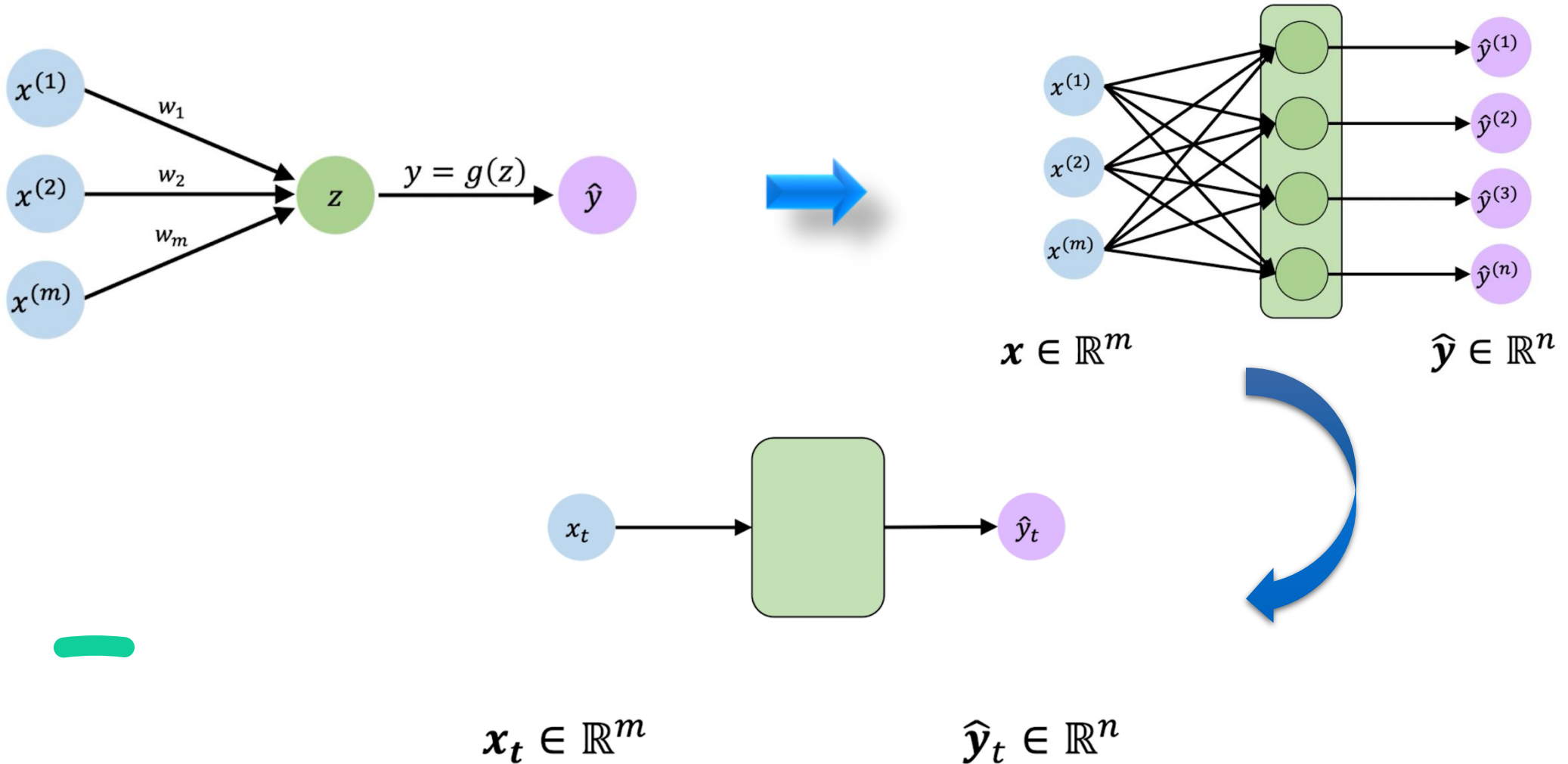


Many to Many
Translation & Forecasting
Music Generation

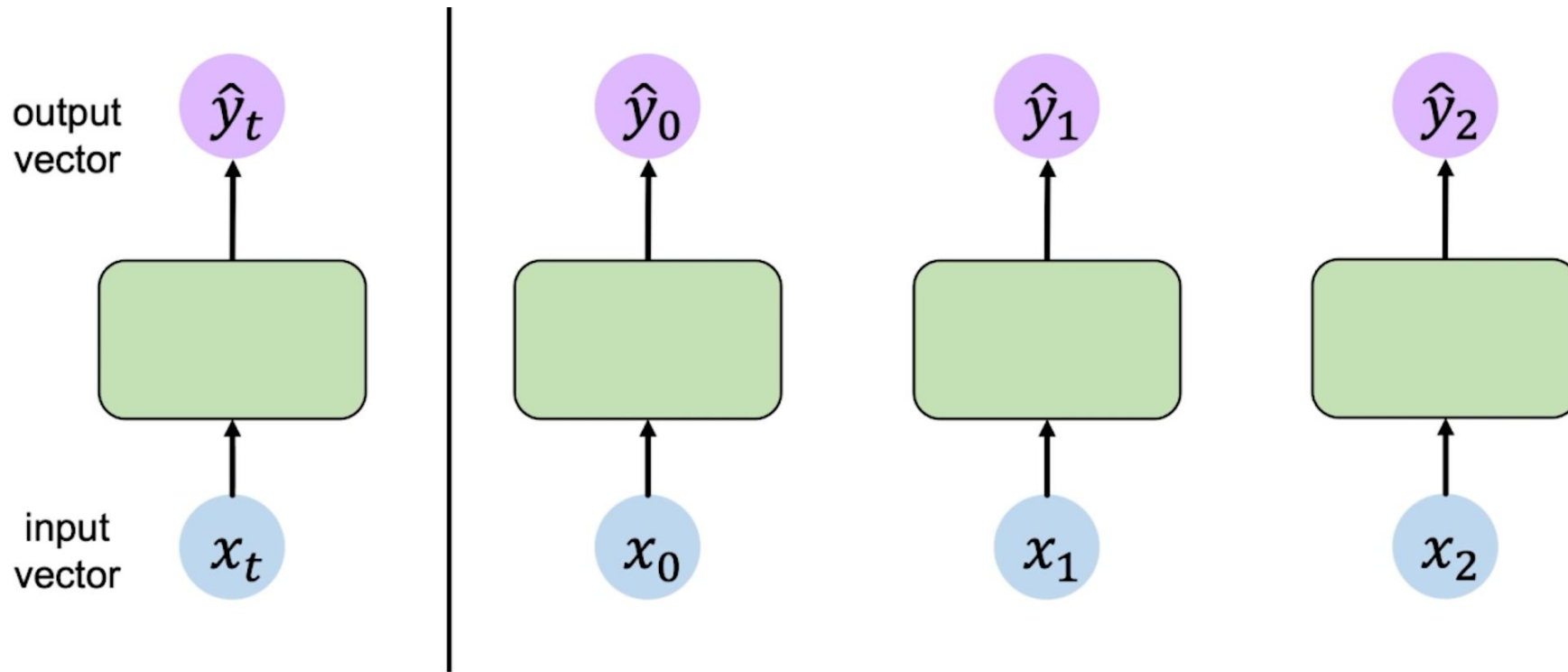
Machine Translation



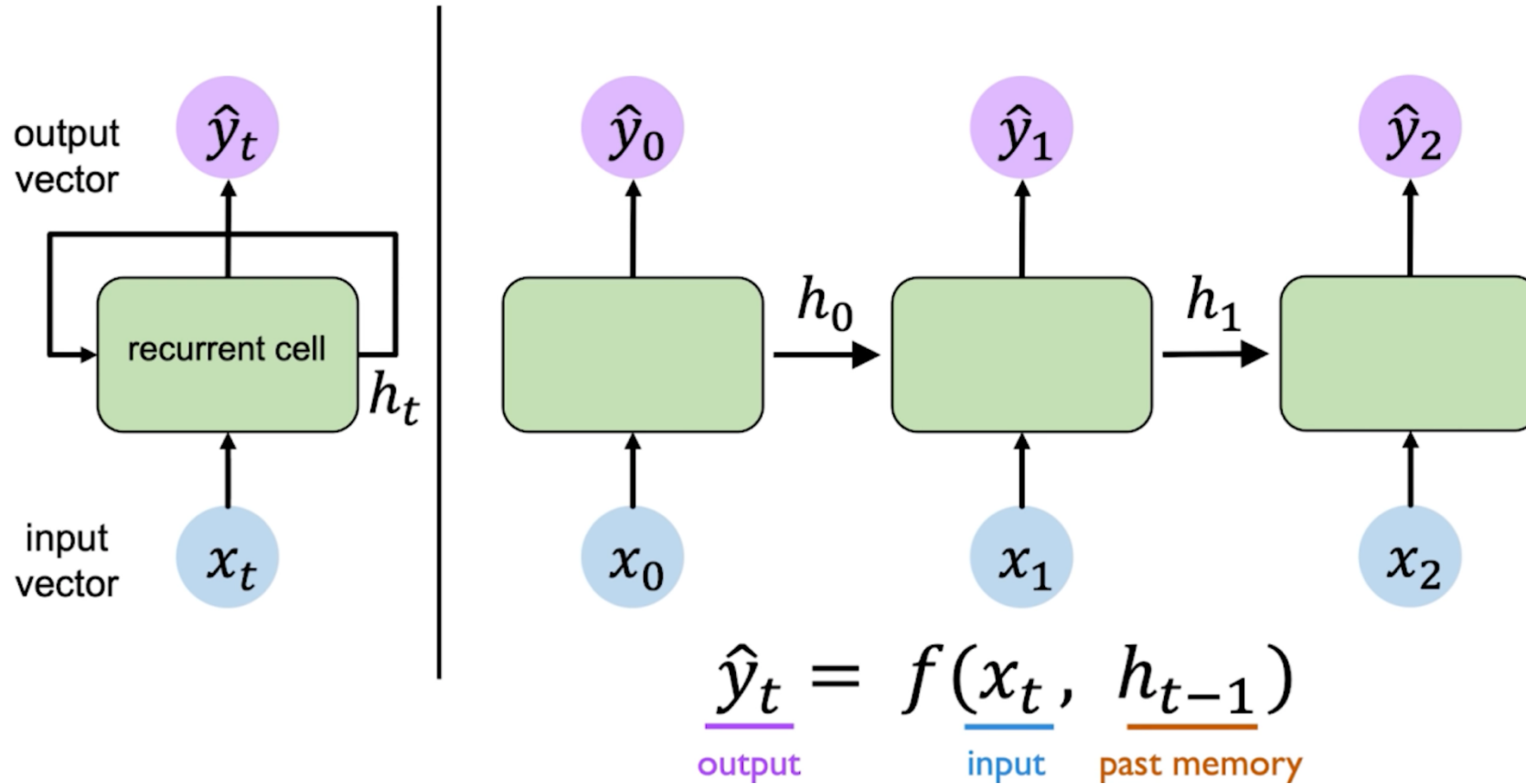
Redes Neuronales (Feed-forward)



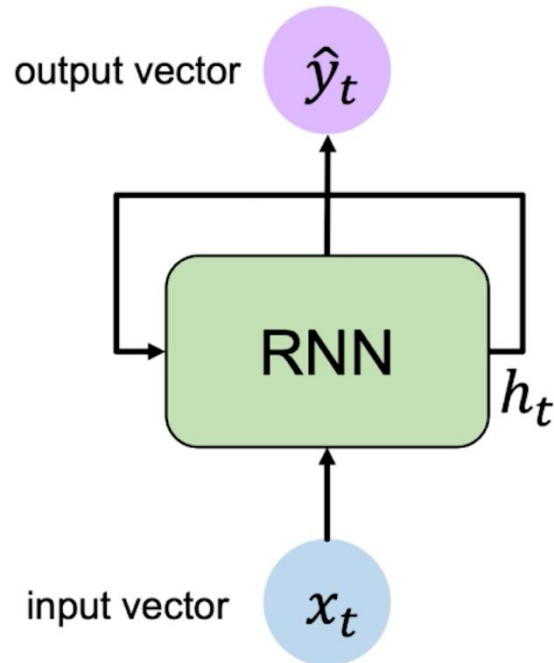
Neuronas con recurrencia



Neuronas con recurrencia (RNNs)



Neuronas con recurrencia (RNNs)



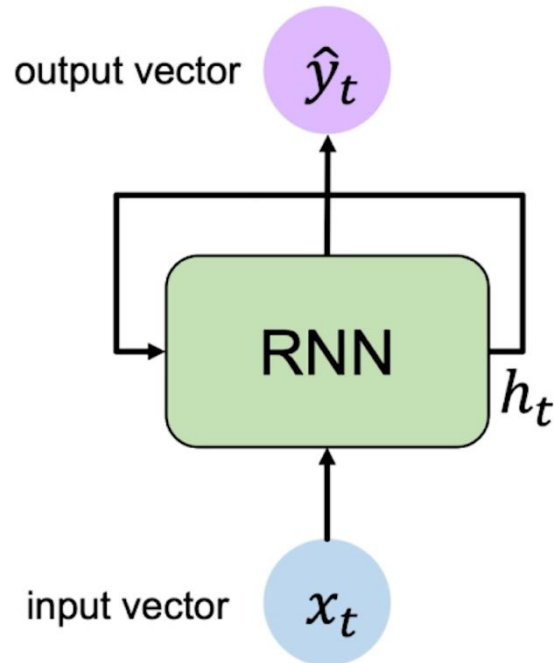
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

Las RNNs tienen un estado h_t , el cual se actualiza en cada iteración cuando la secuencia es procesada

Neuronas con recurrencia (RNNs)



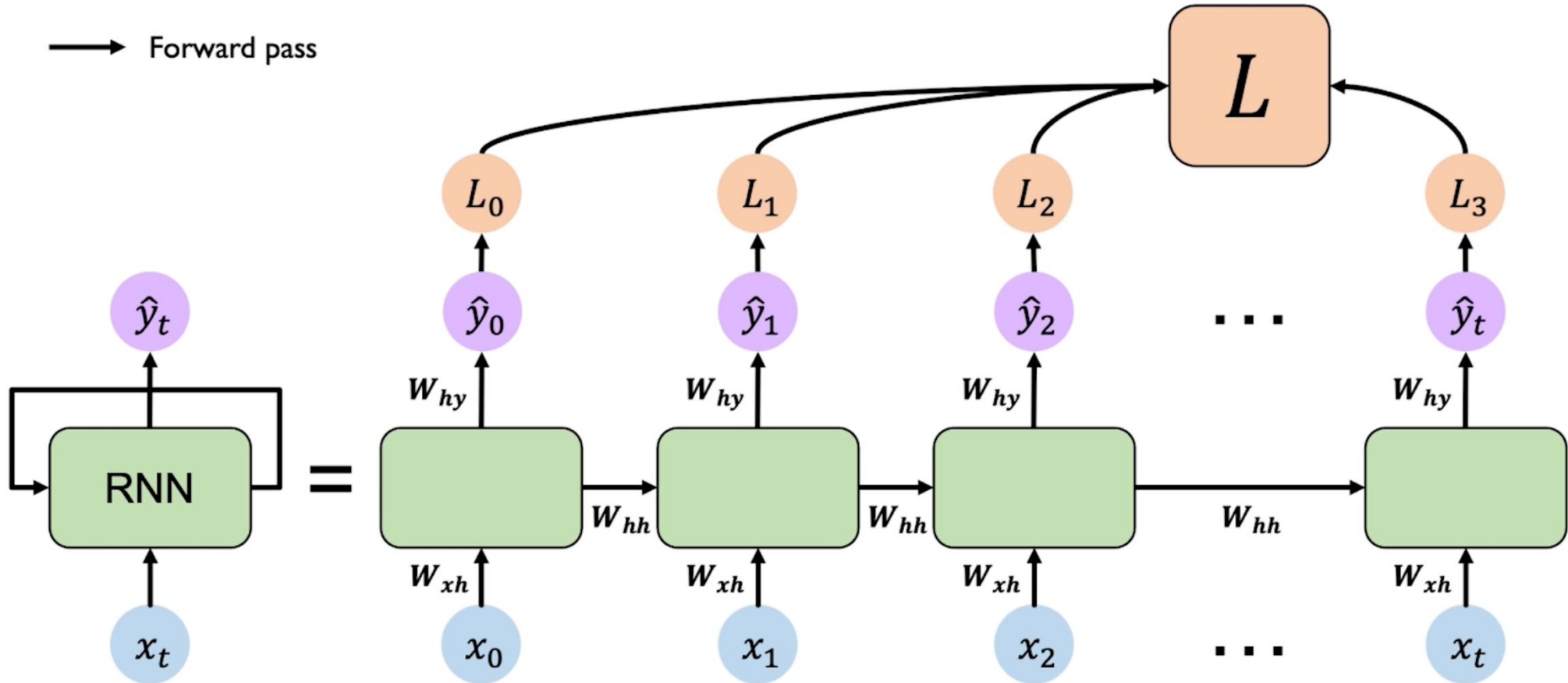
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

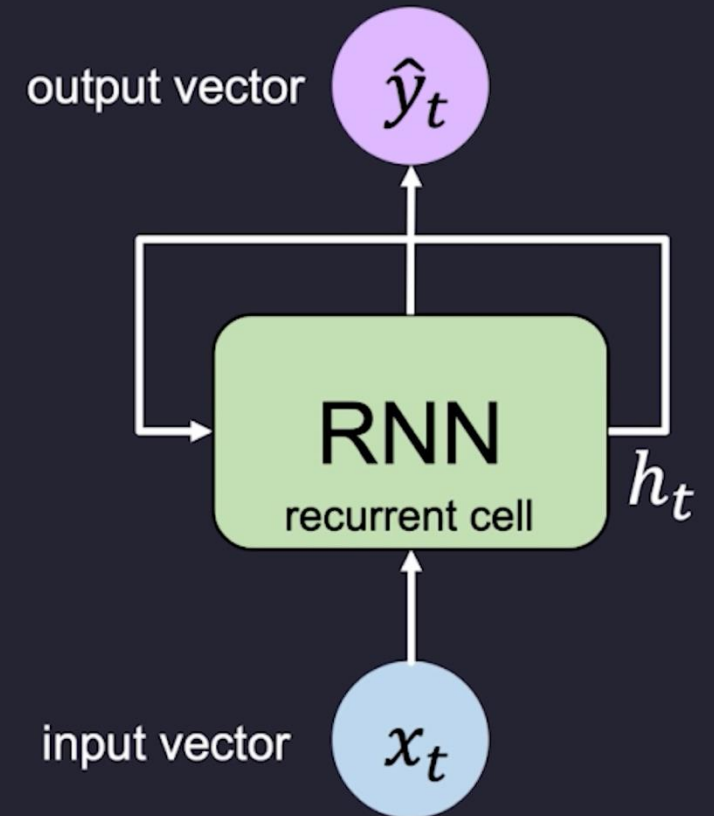
Las RNNs tienen un estado h_t , el cual se actualiza en cada iteración cuando la secuencia es procesada

RNNs: A lo largo del tiempo

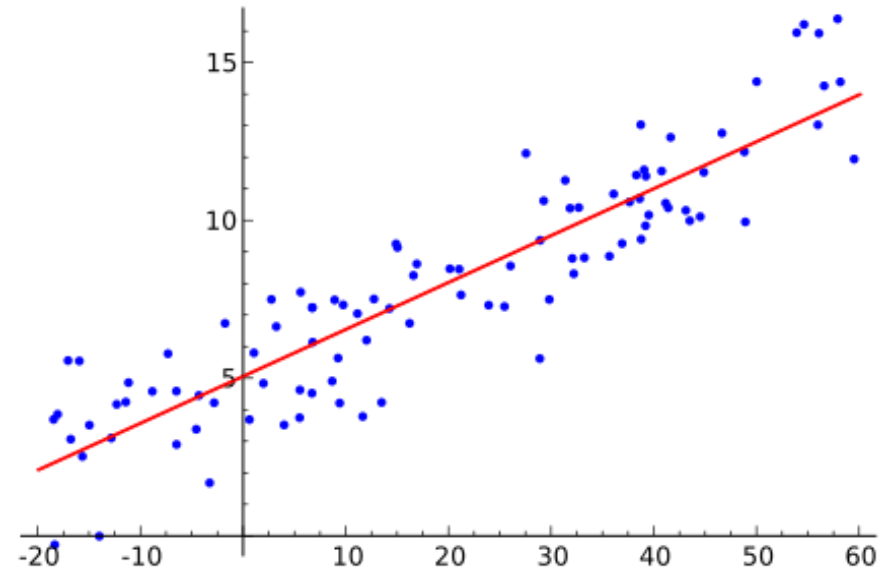


RNNs en Tensor Flow

```
tf.keras.layers.SimpleRNN(rnn_units)
```



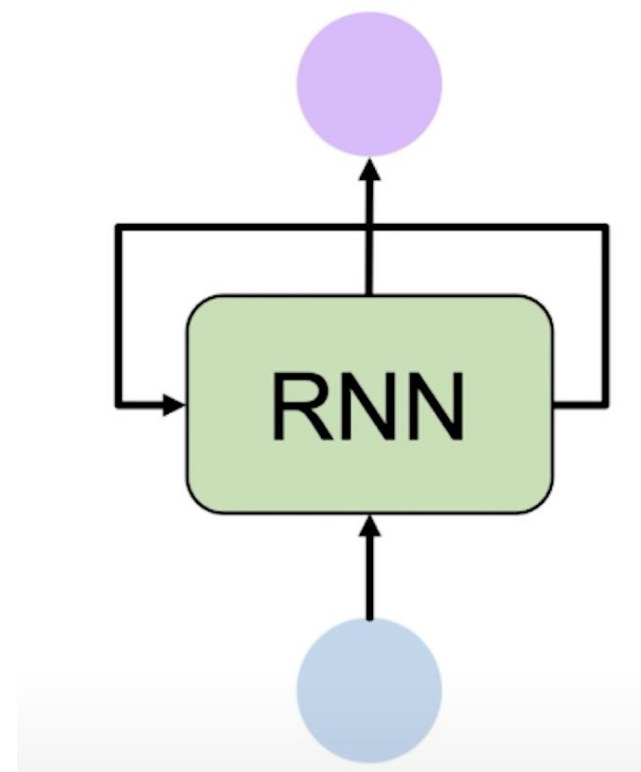
Ejemplo:
Predicción de
siguiente
número



RNNs para el modelado secuencial

- Criterios de diseño:

1. Tener en cuenta la longitud de las secuencias
2. Rastrear dependencias a largo plazo
3. Mantener el orden en las secuencias
4. Compartir parámetros a lo largo de la secuencia



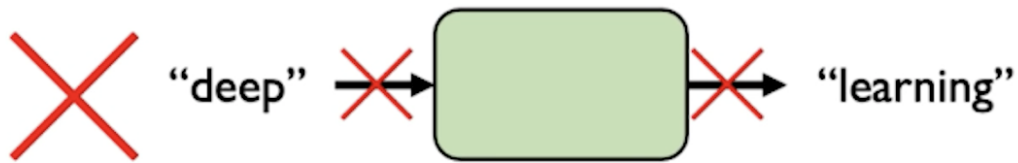
Un problema de modelado de secuencias: Predicción de la siguiente palabra

“This morning I took my cat for a walk.”

given these words

predict the
next word

Representing Language to a Neural Network

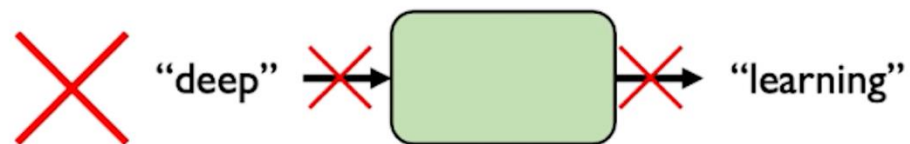


Neural networks cannot interpret words



Neural networks require numerical inputs

Codificando Lenguaje Natural en una RN



Neural networks cannot interpret words



Neural networks require numerical inputs

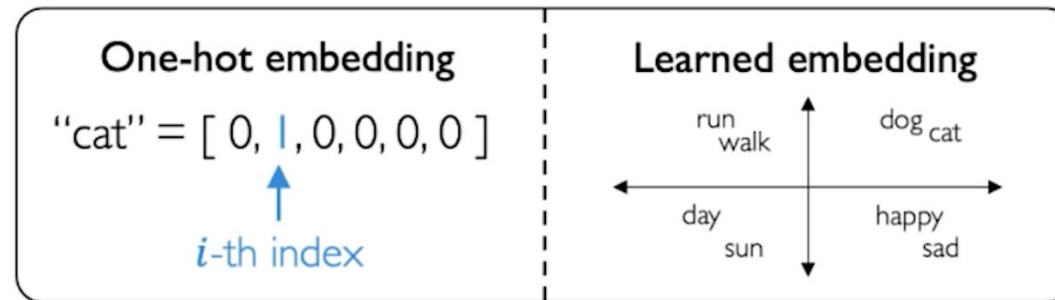
Embedding: transform indexes into a vector of fixed size.

this cat for
my took
a | walk
morning

1. Vocabulary:
Corpus of words

a → 1
cat → 2
... → ...
walk → N

2. Indexing:
Word to index



3. Embedding:
Index to fixed-sized vector

Codificando Lenguaje Natural en una RN

La comida fue muy buena

vs.

Visitamos un restaurante durante la comida

vs.

Esta tarde visitaré al médico debido a que tengo algunos síntomas de gripe

Codificando Lenguaje Natural en una RNN

Diferencias entre secuencias



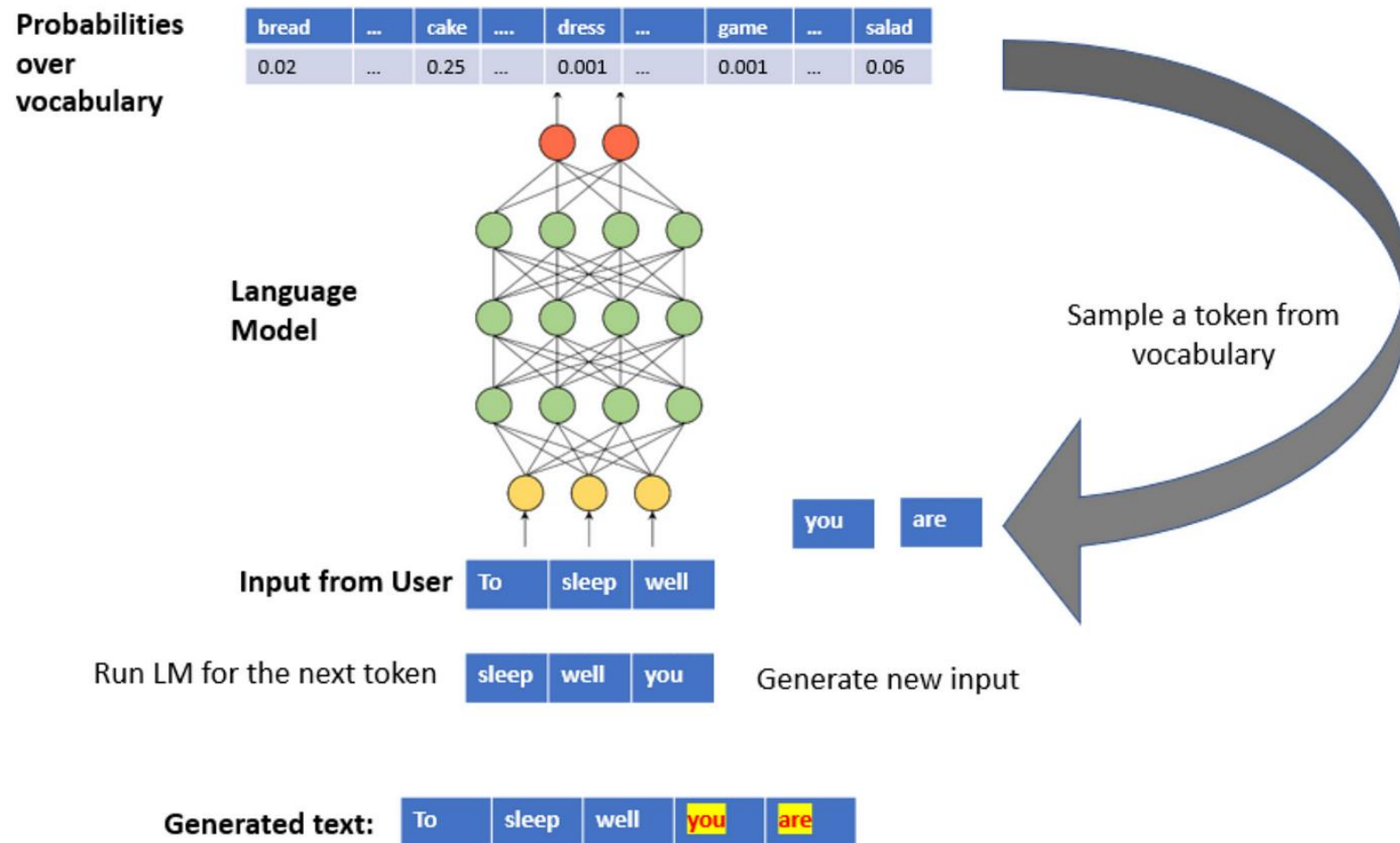
The food was good, not bad at all.

vs.

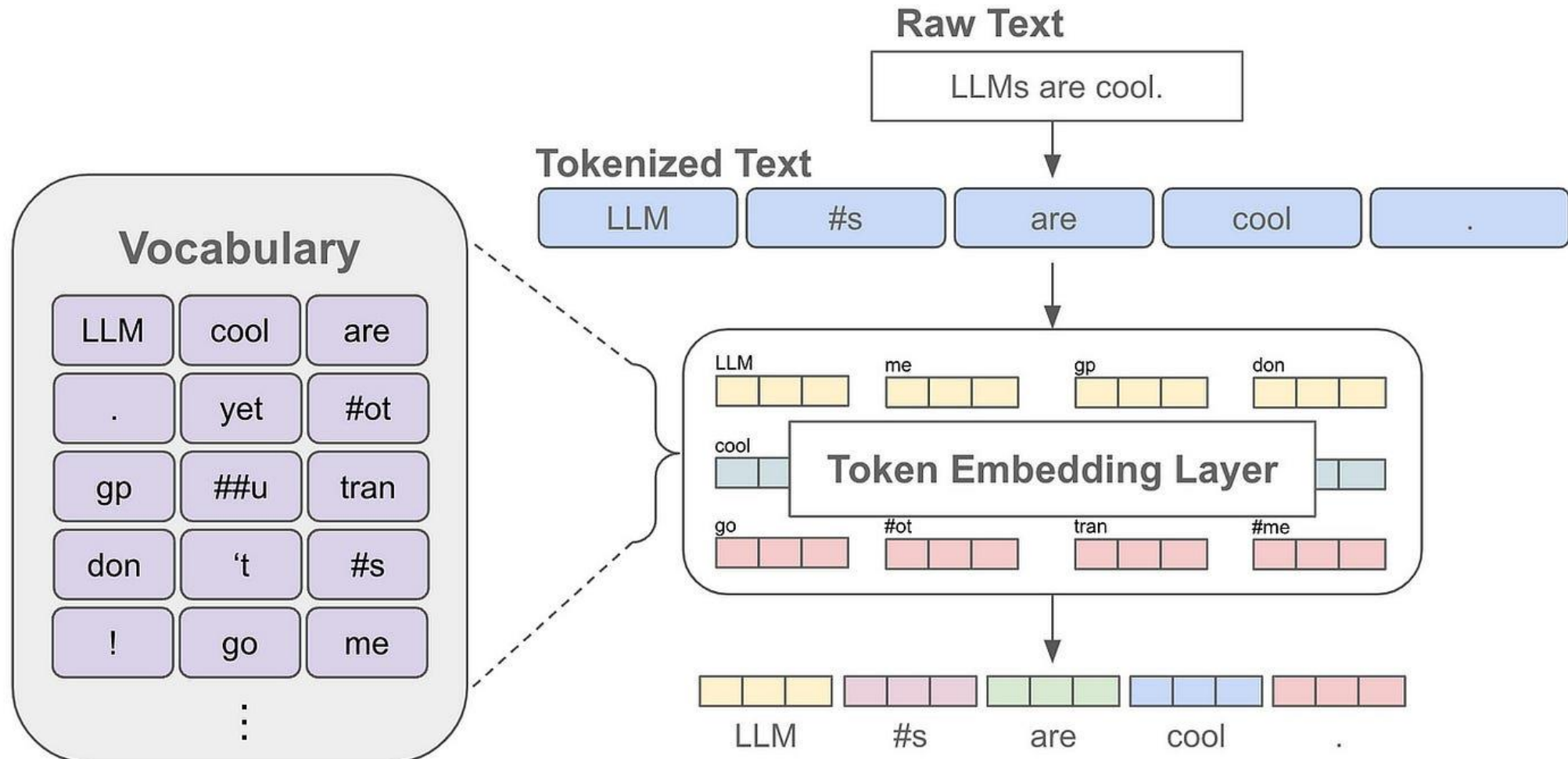
The food was bad, not good at all.



Ejemplo: Predicción de la siguiente palabra



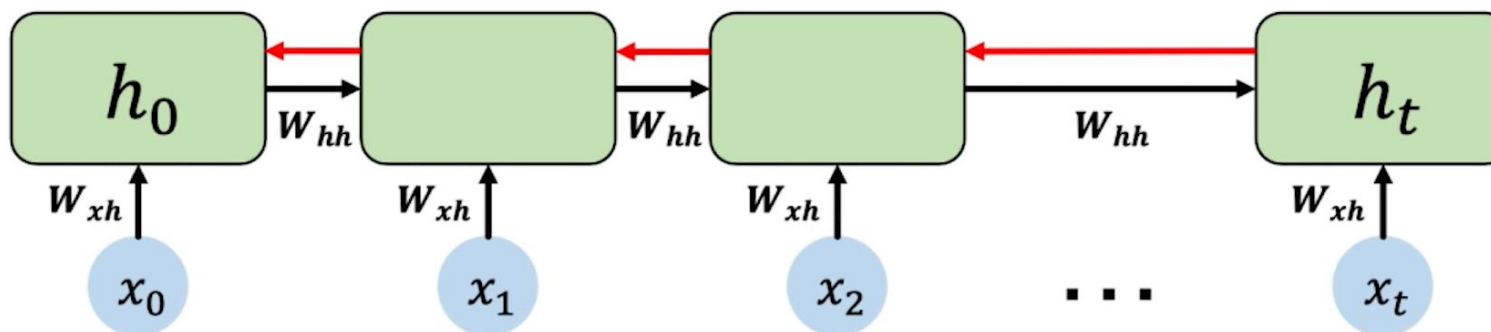
Ejemplo: Predicción de la siguiente palabra





Back Propagation A través del tiempo

Backpropagation: a través del tiempo



Computing the gradient wrt h_0 involves **many factors of w_{hh}** + **repeated gradient computation!**

Many values > 1 :
exploding gradients

Gradient clipping to
scale big gradients

Many values < 1 :
vanishing gradients

1. Activation function
2. Weight initialization
3. Network architecture

El problema de las dependencias a largo plazo

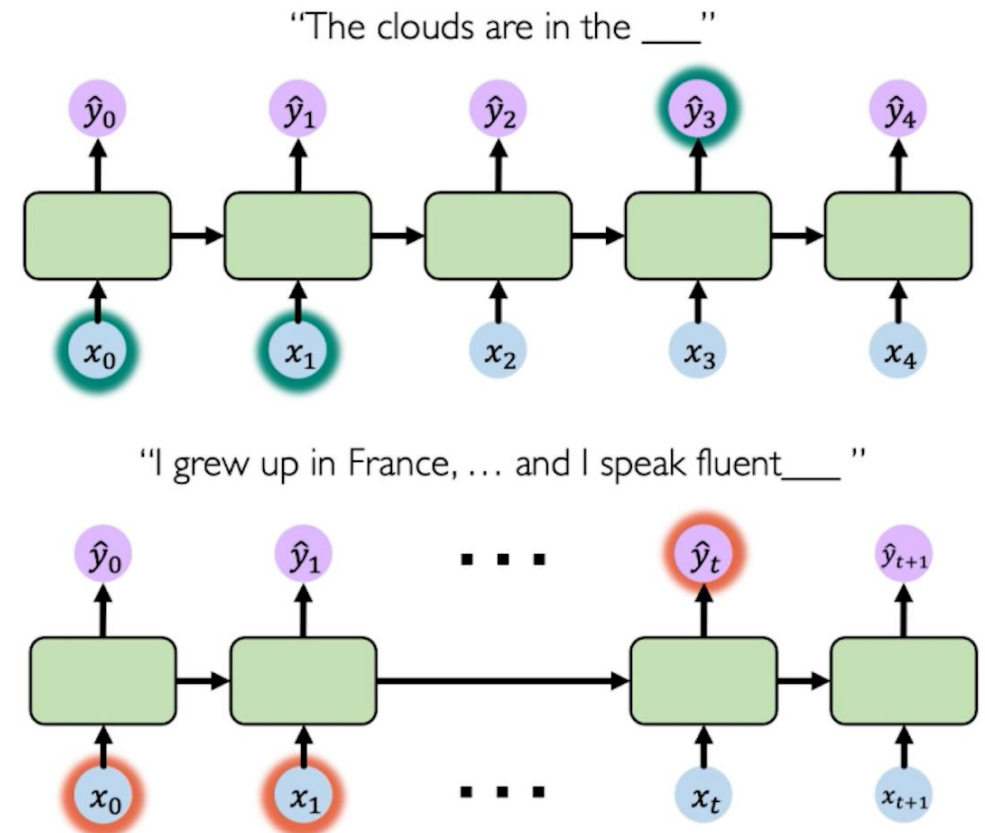
¿Porqué las dependencias a largo plazo son un problema?



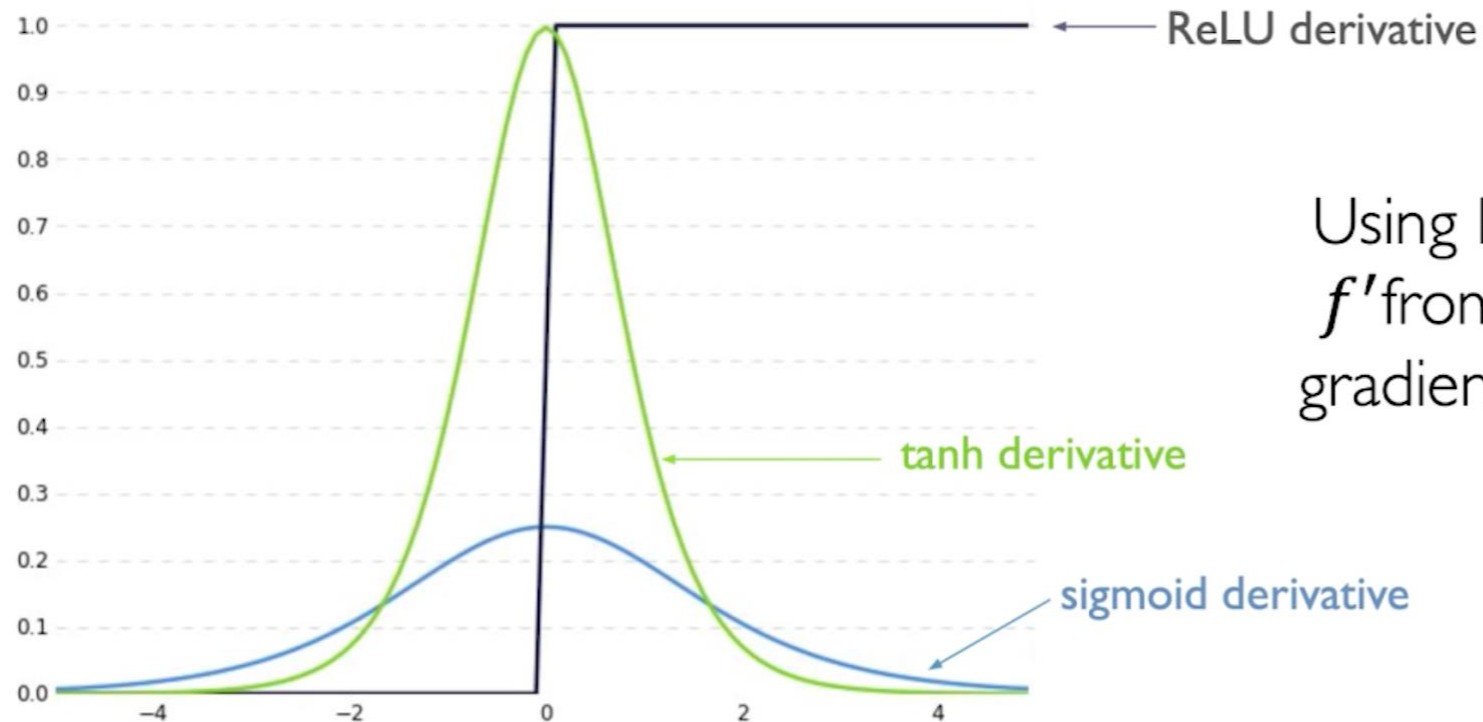
Multiplican múltiples números pequeños



Esta propagación genera gradientes cada vez más pequeños



Posible solución 1: Funciones de activación



Using ReLU prevents f' from shrinking the gradients when $x > 0$

Posible solución 2:

Inicialización de parámetros

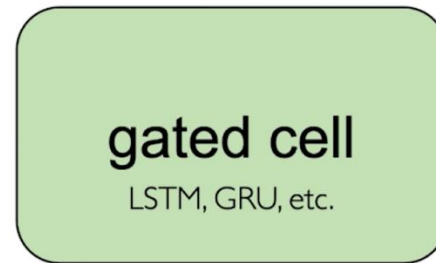
- Inicializar pesos usando una matriz identidad
 - Inicializar bias con cero
- Ayuda a prevenir que los pesos se **desvanezcan**.

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

Posible solución 3:

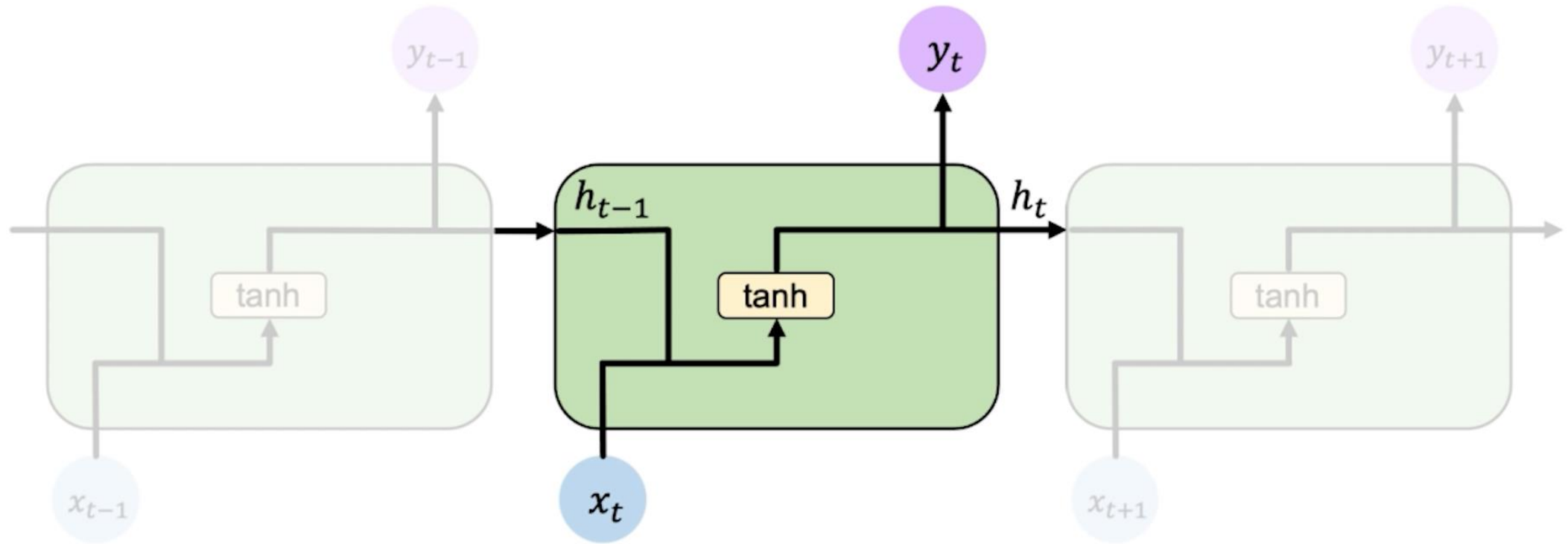
Celdas con compuertas

- Utilizar unidades recurrentes con compuertas para controlar que información fluye por la celda.

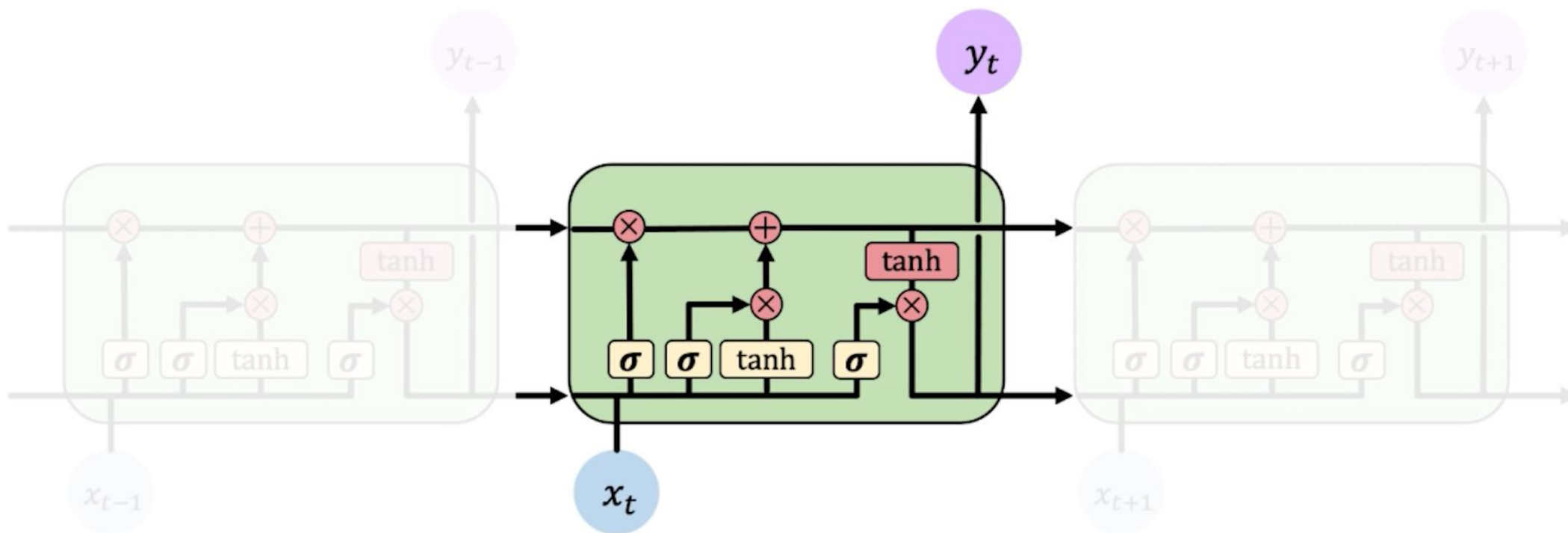


- Memoria de corto y largo plazo (LSTMs), las redes se basan e celdas con compuertas para registrar información a lo largo de múltiples pasos.

Redes Neuronales Recurrentes Estándar



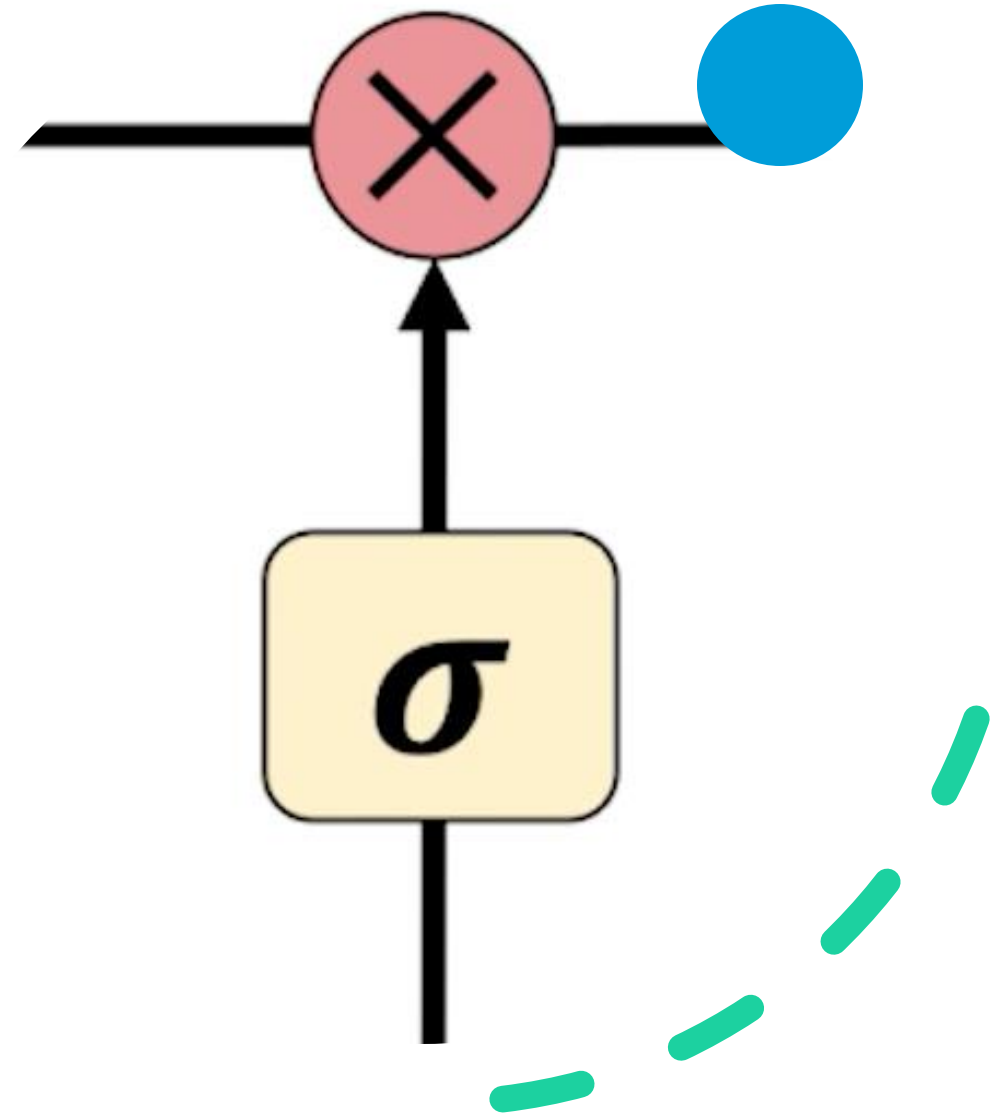
Long Short Term Memory (LSTMs)



```
tf.keras.layers.LSTM(num_units)
```

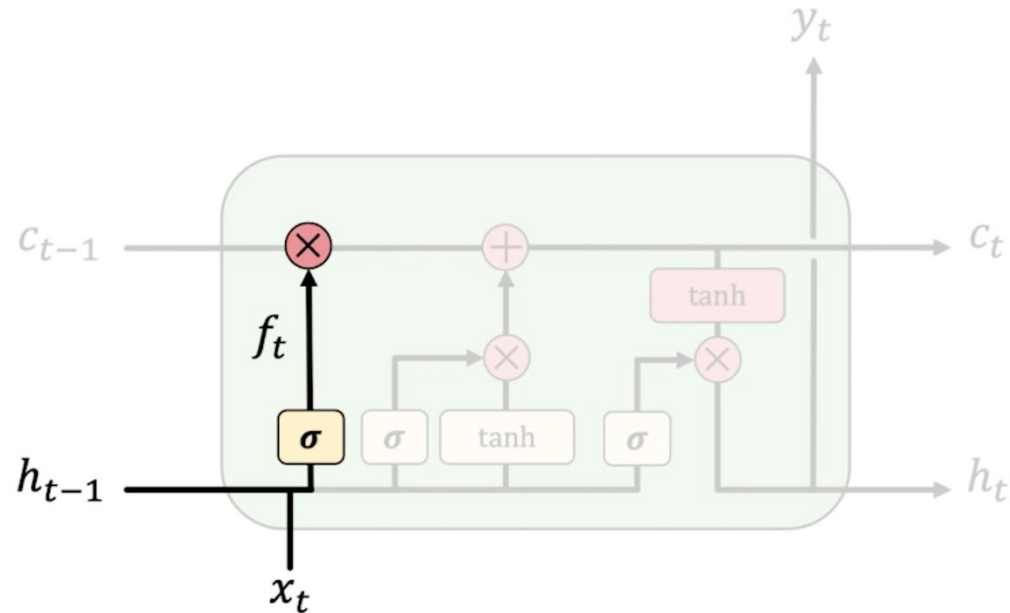
Long Short Term Memory (LSTMs)

- La información se agregar o elimina a través de estructuras llamadas compuertas.
- Las compuertas dejan fluir la información, este se puede lograr a través de una función sigmoide.



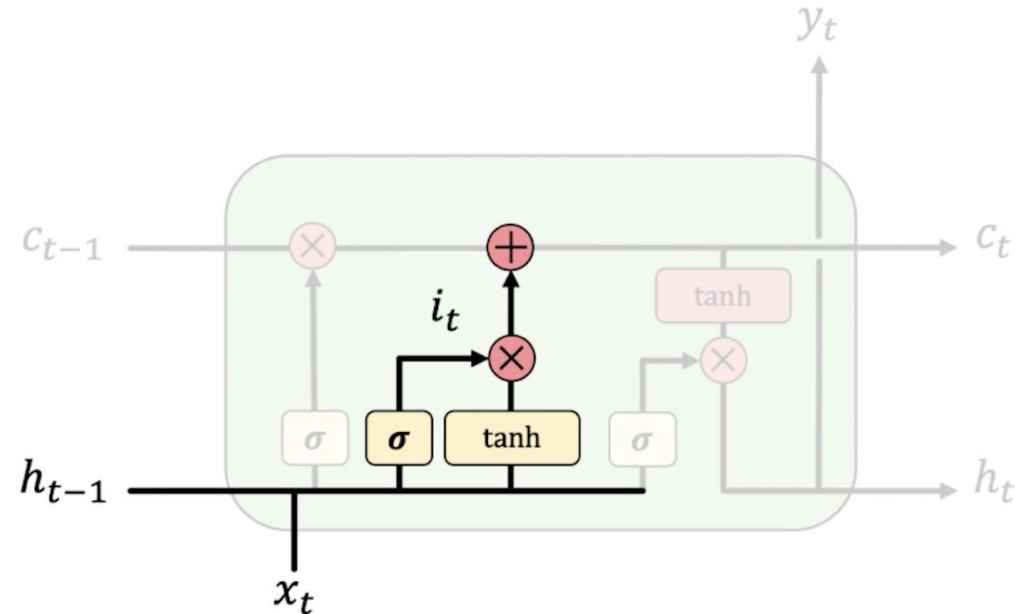
Long Short Term Memory (LSTMs)

1) **Forget** 2) Store 3) Update 4) Output
LSTMs **forget irrelevant** parts of the previous state



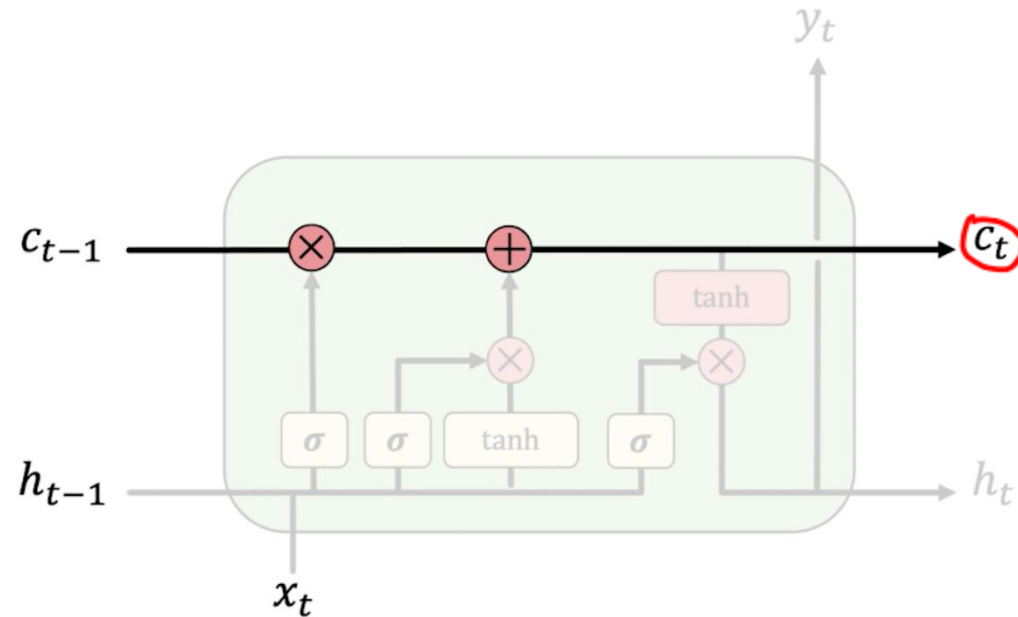
Long Short Term Memory (LSTMs)

1) Forget **2) Store** 3) Update 4) Output
LSTMs **store relevant** new information into the cell state



Long Short Term Memory (LSTMs)

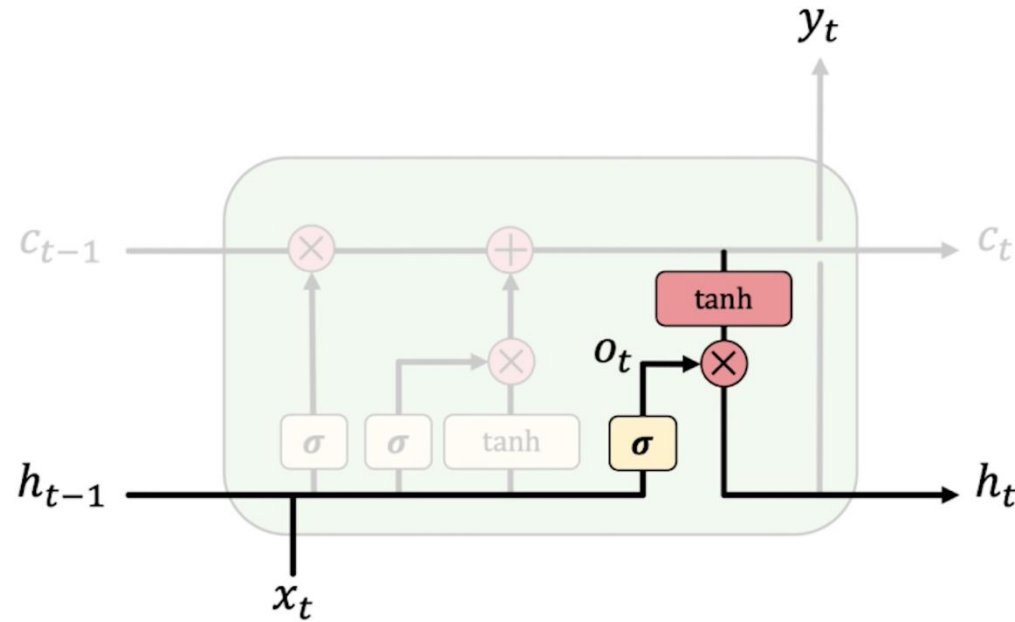
1) Forget 2) Store **3) Update** 4) Output
LSTMs **selectively update** cell state values



Long Short Term Memory (LSTMs)

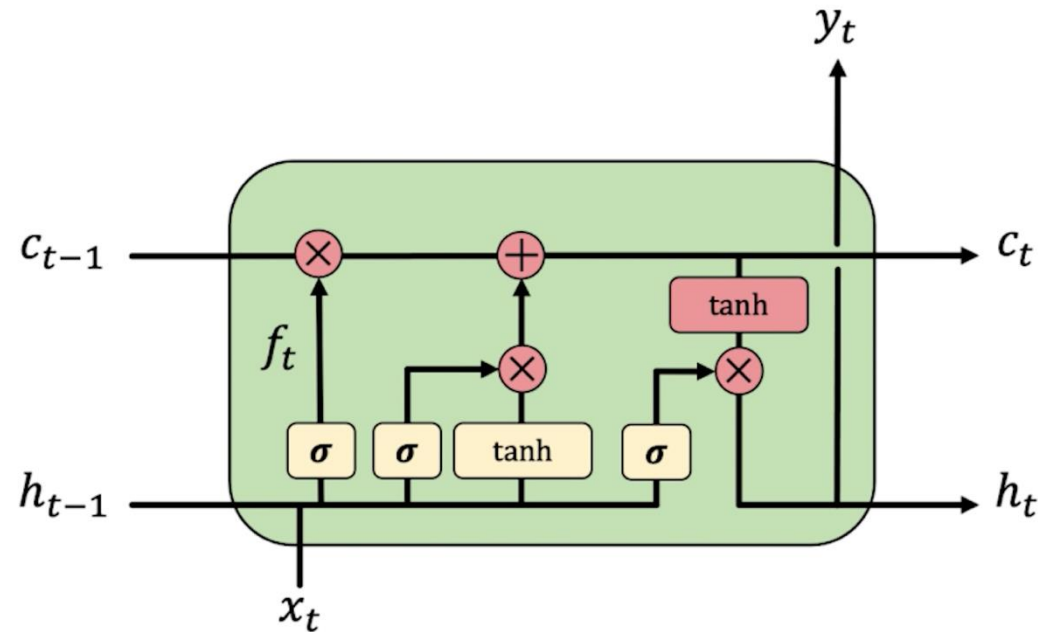
1) Forget 2) Store 3) Update 4) **Output**

The **output gate** controls what information is sent to the next time step



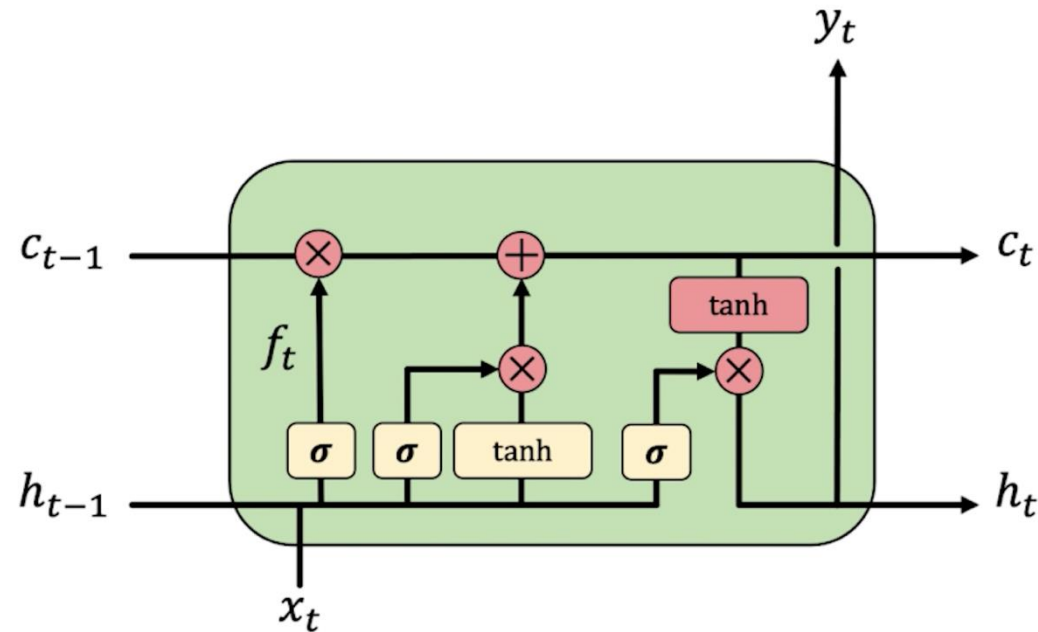
Long Short Term Memory (LSTMs)

1) Forget 2) Store 3) Update 4) Output

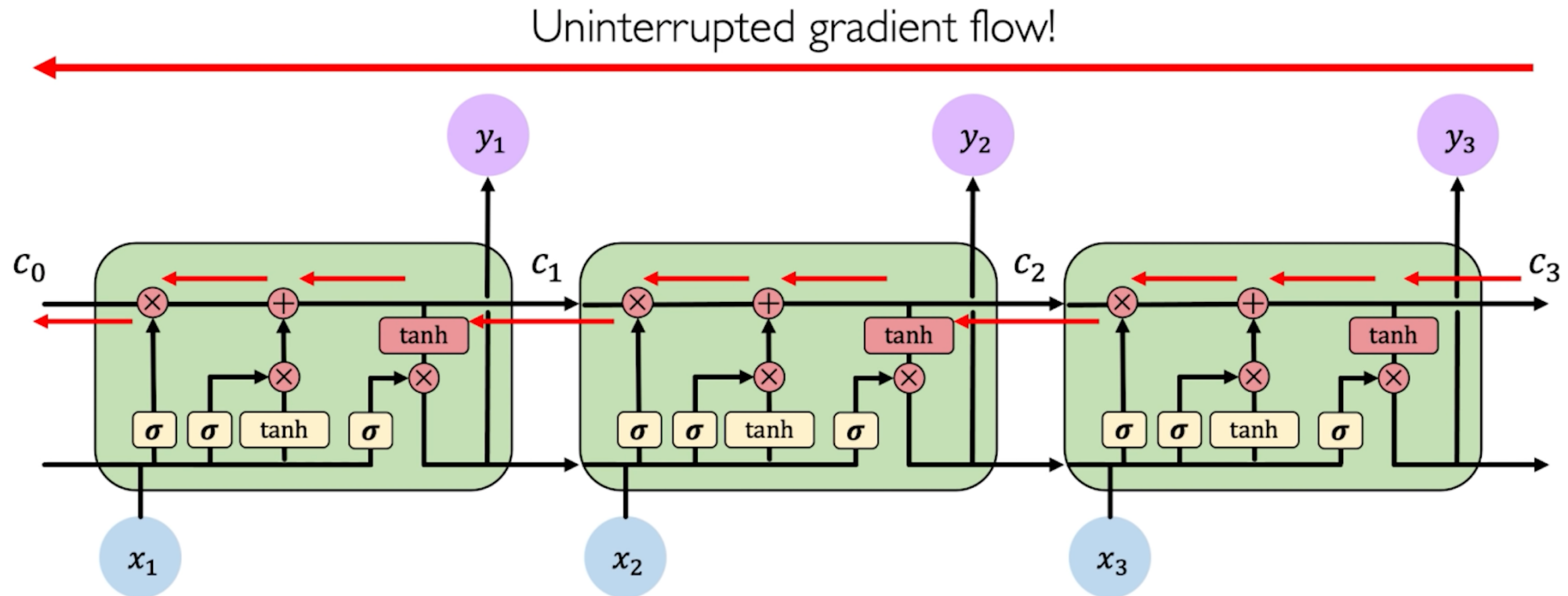


Long Short Term Memory (LSTMs)

1) Forget 2) Store 3) Update 4) Output



Long Short Term Memory (LSTMs)



LSTMs: Conceptos principales

1. Mantienen un estado separado de las salidas
2. Utiliza compuertas para controlar el flujo de información
 - Las compuertas eliminan información irrelevante
 - Almacenan información relevante de la entrada actual
 - Actualizan de forma selectiva el estado de una celda
 - Las compuertas de salida retornan una versión filtrada del estado de la celda.
3. Backpropagation con uninterrupted gradient flow

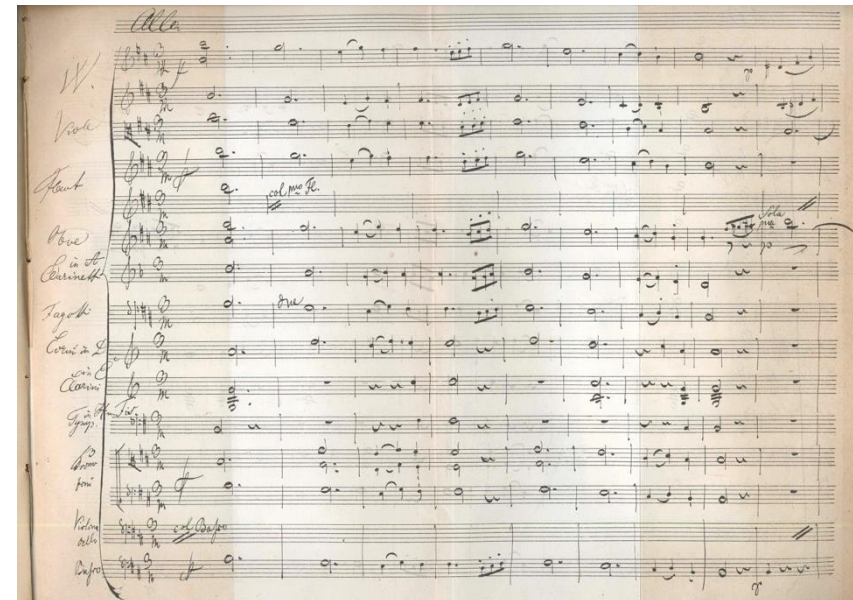
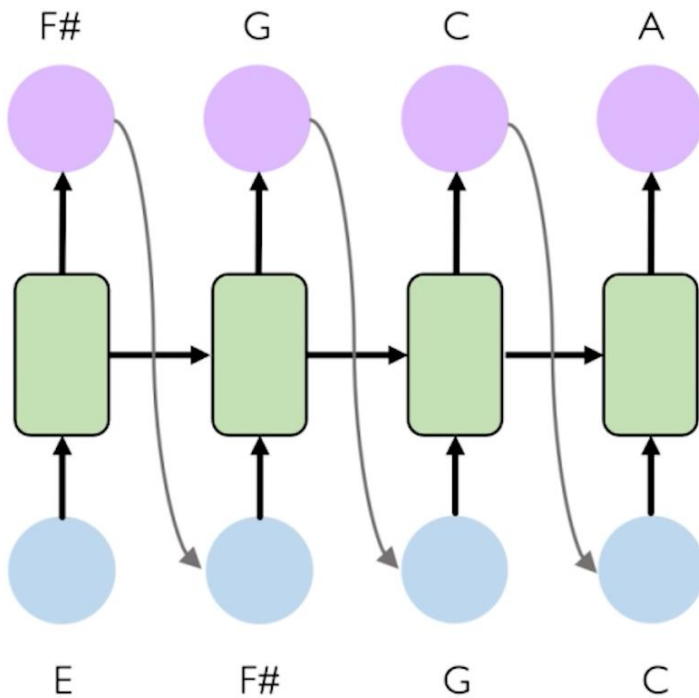
Aplicaciones de RNN

Ejemplo: Generación de música



Sinfonía Incompleta de Schubert

- **Entrada:** Conjunto de notas musicales
- **Salida:** Siguiente nota musical



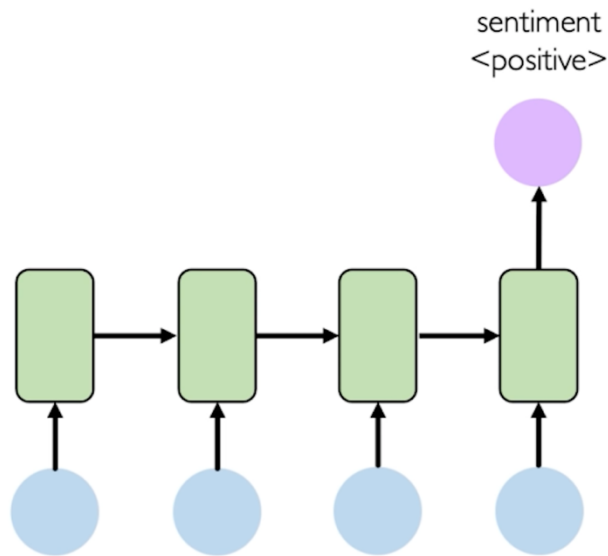
https://www.youtube.com/watch?v=RCo8Feho1RI&ab_channel=MusicalSeries

Listening to
1st movement

Sinfonía incompleta Schubert



Ejemplo: Clasificación de sentimientos



- **Entrada:** Secuencia de palabras
- **Salida:** Probabilidad de obtener un sentimiento positivo.

Problema de clasificación

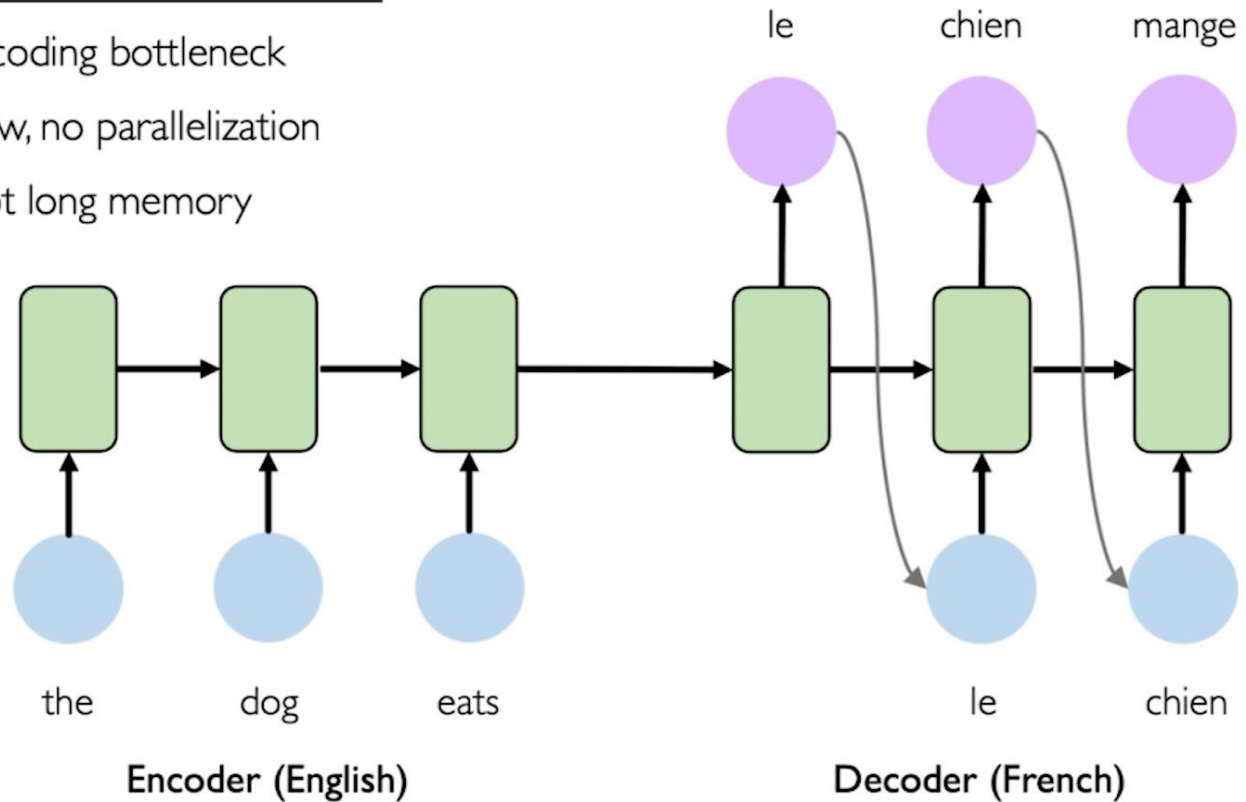


```
loss = tf.nn.softmax_cross_entropy_with_logits(y, predicted)
```

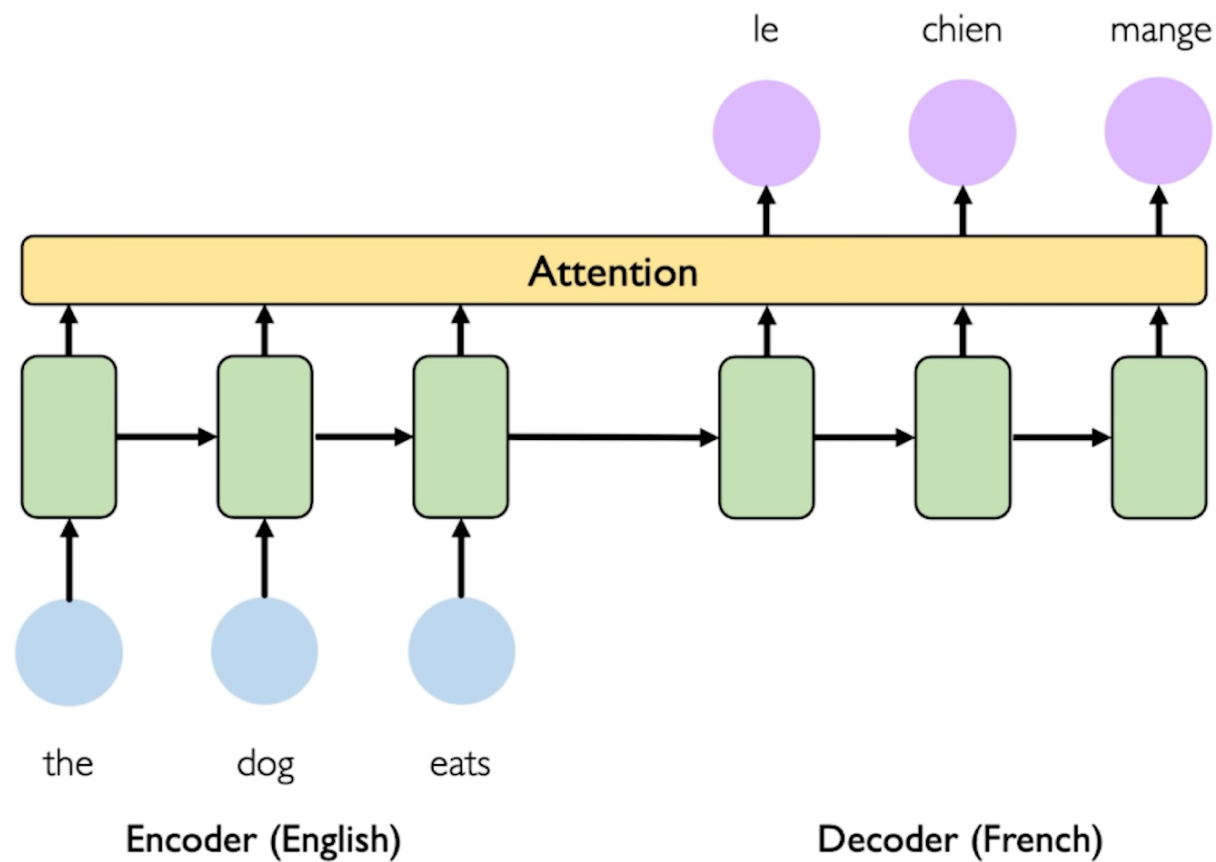
Traducción automática

Potential Issues

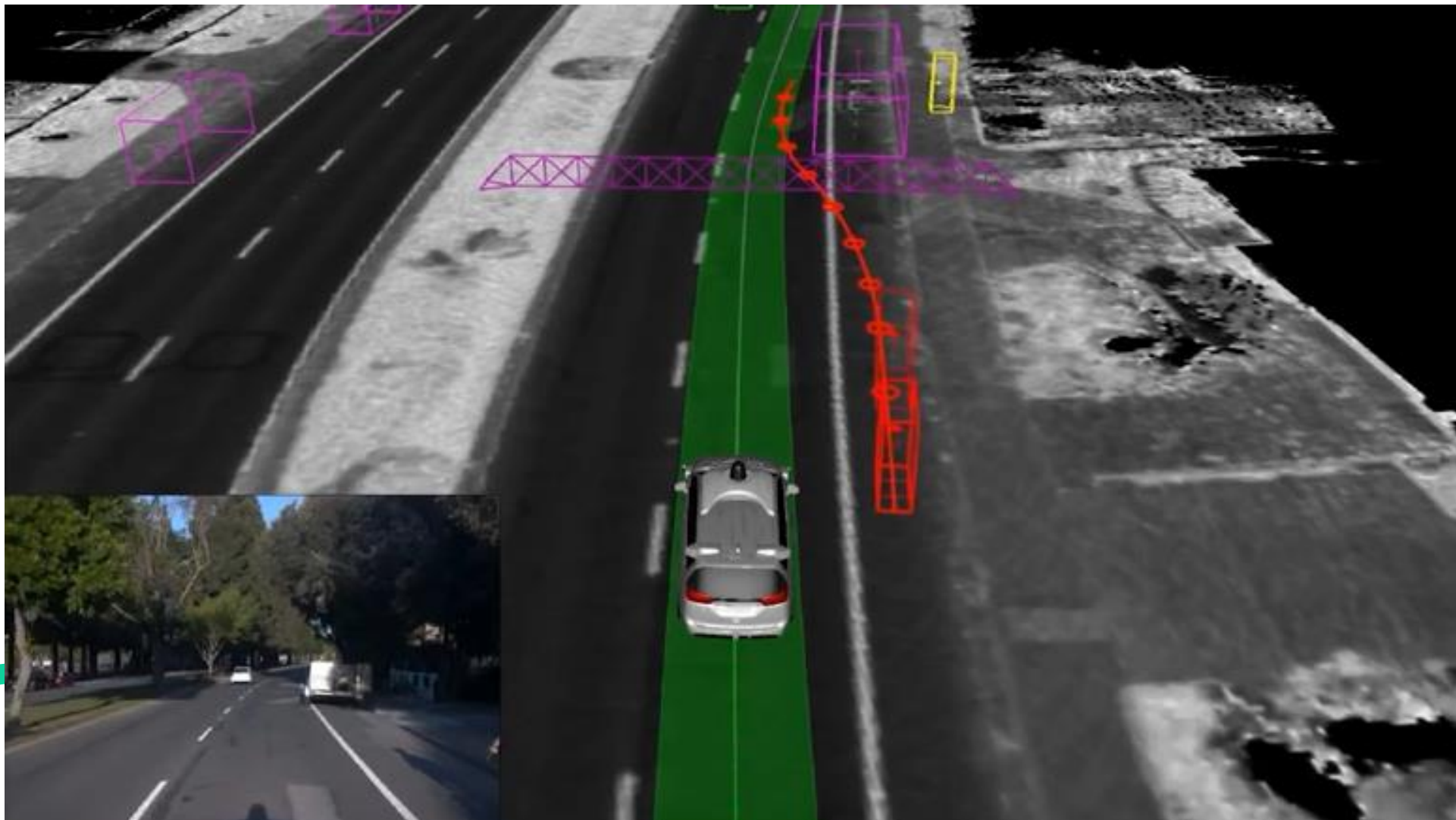
-  Encoding bottleneck
-  Slow, no parallelization
-  Not long memory



Traducción automática



Predicción de trayectorias



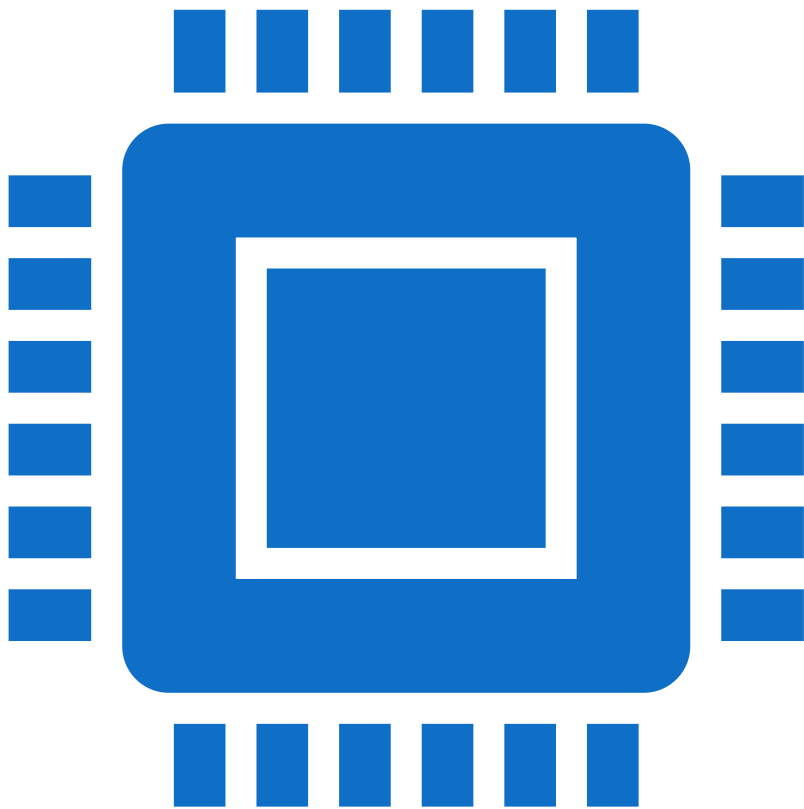
Procesamiento del Lenguaje Natural (PLN)

1. **Análisis de Sentimiento:** Las RNN pueden analizar el sentimiento de las frases teniendo en cuenta la secuencia de palabras.
2. **Traducción Automática:** Pueden traducir frases de un idioma a otro.
3. **Reconocimiento de Entidades Nombradas:** Las RNN pueden identificar entidades como nombres, lugares y organizaciones en texto.
4. **Reconocimiento de Voz:** Pueden convertir el lenguaje hablado en texto.

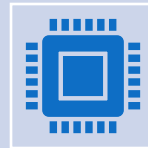


Procesamiento de Señales

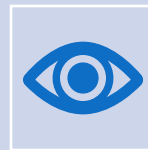
- 1. Síntesis de Audio:** Pueden generar música u otras formas de audio basándose en muestras previas.
- 2. Descripción de Imágenes:** Las RNN se pueden combinar con redes neuronales convolucionales (CNN) para generar descripciones textuales de imágenes.



Robótica



Planificación de Rutas: Las RNN pueden analizar una secuencia de datos de sensores para ayudar en la navegación de robots.



Reconocimiento de Gestos: Pueden reconocer e interpretar gestos humanos para una interacción humano-máquina más natural.



Atención Médica

1. Monitoreo de Pacientes: Las RNN pueden analizar datos de series temporales de sistemas de monitoreo de pacientes para proporcionar señales de alerta temprana.

2. Diagnóstico Médico: Pueden interpretar secuencias de imágenes médicas para ayudar en el diagnóstico.



Finanzas

- 1.Comercio Algorítmico:** Las RNN pueden emplearse para tomar decisiones de compra o venta basadas en una secuencia de indicadores del mercado financiero.
- 2.Evaluación de Crédito:** Pueden evaluar el riesgo de prestar a individuos o entidades analizando datos financieros históricos.

Juegos y Entretenimiento



IA en Juegos: Las RNN pueden modelar el comportamiento de los jugadores o entidades del juego a lo largo del tiempo.



Recomendación de Contenido: Pueden analizar la secuencia de interacciones del usuario para recomendar contenido relevante.

Otros

01

Vehículos Autónomos:

Las RNN pueden procesar secuencias de datos de sensores para mejorar la navegación y toma de decisiones.

02

Optimización de la Cadena de Suministro:

Pueden analizar datos históricos para mejorar la gestión de inventario.